Measuring Privacy-Preserving Computation in One Place: MPC, FHE, ZKP and Their Hybrids

(author names elided for review)

Abstract

Privacy-preserving computation is not one field but four—secure multi-party computation (MPC), fully homomorphic encryption (FHE), garbled circuits, and zero-knowledge proofs (ZKP)—each with its own literature, benchmarks and accounting conventions. A practitioner choosing between them, or a reviewer assessing a claim that one beats another, must assemble numbers from different papers, measured on different hardware, over different networks, counting different things. Cross-paradigm comparison is therefore largely *citation assembly* rather than measurement.

We take a different route. We survey the paradigms through a single artifact, Baru, in which one source program runs on all of them—a plain interpreter, three-party replicated-secret-sharing MPC, garbled circuits, BFV/BGV/TFHE homomorphic encryption, a Binius64 SNARK, and *combinations* in which values cross between representations mid-computation without being exposed. Every backend is driven through the same transport, charged by the same byte and round accounting, and checked bit-exactly against the same plaintext reference. A network cost model is validated against a real socket to within 13.6 % worst case, so a claim about a wide-area deployment is computed from measured counts rather than asserted.

Several widely repeated intuitions survive re-measurement; several do not. Among the results that only a single-harness study can produce: the FHE-versus-MPC crossover is a property of *the baseline's preprocessing* rather than of either cryptosystem, and disappears once MPC is given somewhat-homomorphic preprocessing; active security on the nonlinear surface of private inference costs 2.63× bytes but only 1.06× rounds—and 9.70× bytes once the semi-honest baseline it is measured against is allowed to pack, which only the unverified side can do. We also report our own withdrawn claims—a ZKP backend that proved nothing, a bit-decomposition that leaked its inputs, and a comparison-protocol crossover whose location we got wrong three times before measuring it—because a study that reports only what worked is not reporting.

1 The comparability problem

If we ask how fast private inference is, the literature answers in units that cannot be compared. Gazelle [35], Delphi [45] and Cheetah [32] report two-party hybrid systems in which the server holds the model in plaintext. Falcon [55] reports three-party replicated sharing on a LAN. Papers on CKKS [15] report amortized throughput over batched slots, and papers on SNARKs report prover time for a circuit that another paper would express with a different constraint count. Each of these measurements is internally rigorous, but taken together they do not compose. In this section we describe the obstacles that prevent them from composing, and we state what we propose to do about it.

We remark that four obstacles recur.

1. **Different hardware.** A 5× difference between two systems is unfalsifiable when one ran on a server-class Xeon and the other on a laptop.

2. **Different accounting.** "Communication" may mean payload bytes, serialized bytes, or bytes per party. "Rounds" may mean protocol rounds, messages, or interactive calls. These differ by more than the effects being measured.

3. **Different security models.** Semi-honest and malicious figures appear in adjacent columns of secondary literature without the distinction surviving the citation.

4. **Different functions.** Two ReLU costs may be for different bit widths, different fixed-point scales, or—in the worst case we found in our own work—subtly different functions that nobody checked against a common reference.

A survey that is assembled from such numbers inherits all four obstacles. The alternative that we pursue in this work is to re-measure the paradigms inside a single harness, in which the obstacles are controlled by construction: the same machine, the same accounting code, machine-checked security levels, and a bit-exactness assertion against a shared plaintext reference before any cost is quoted.

The cost of this approach is equally clear, and we state it at the outset. Our implementations are research-quality Rust and not the tuned C++ of the systems that we discuss, and therefore **the absolute figures given here are not competitive with optimized implementations and should not be read as such**. What a single harness buys us is *internally valid ratios*, that is, comparisons in which the confounding factors listed above are held fixed. It means that our internal comparisons are stronger evidence than our external ones, and where we compare against a published number we say so explicitly and treat the comparison accordingly.

Organization. In Section 2 we delimit the scope of the survey. In Section 3 we give the cryptographic preliminaries, that is, the definitions, theorems and algorithms on which the remainder depends. In Section 4 we organize the four paradigms by what each one is expensive in. In Section 5 we describe our artifact and the abstract machine at its centre, and in Section 6 we survey and measure each paradigm in turn. In Section 7 we position this study with respect to the literature, and in particular with respect to MP-SPDZ. In Section 8 we present the methodology, and in Section 9 the cross-paradigm findings that motivate it. In Section 10 we give a case study on private medical image diagnosis. In Section 11 we record what the methodology caught in our own work, in Section 12 the threats to validity, and in Section 13 the problems that remain open.

Contributions.

- A **systematization** of four paradigms organized by what they actually cost, with the cost dimensions (rounds, bytes, compute, preprocessing) separated rather than collapsed into one "performance" number (Section 4).

- A **single-artifact measurement study**: roughly 70,000 lines of Rust in which one program runs on every paradigm, with a validated network model and uniform accounting (Sections 5 and 8).

- **Cross-paradigm findings** not visible from any single paradigm's literature (Section 9), including two crossovers whose location depends on parameters the original claims left implicit—one of which we ourselves mislocated three times before measuring it (Section 9.3).

- A **methodology for honest benchmarking** (Section 8), with the record of what it caught in our own work (Section 11).

2 Scope

In this section we delimit what the survey covers, and we state the workload that we use throughout.

In scope. Semi-honest and malicious three-party MPC over rings; garbled circuits in the honest-majority setting; RLWE-based FHE (BFV [29], BGV [8]) and TFHE [17]; FHE↔MPC scheme switching; SNARKs over 64-bit words (Binius64 [25]); and the supporting primitives these need—oblivious transfer, private set intersection, and Beaver-triple [4] preprocessing from both OT and somewhat-homomorphic encryption (SHE).

Out of scope. Differential privacy and federated learning (a different threat model: they protect an *aggregate*, not a computation); trusted execution environments (hardware trust assumption); ORAM (surveyed but not implemented, see Section 13); and transformer nonlinearities such as softmax and LayerNorm, which need exponential, reciprocal and inverse-square-root subprotocols we do not have.

Application focus. We use private neural-network inference as the running workload, because it is the one application that exercises every paradigm at once. It requires dense linear algebra, where the packing techniques of homomorphic encryption are effective; it requires comparisons, where secret sharing and garbled circuits compete; and it admits verifiable computation, where succinct proofs apply. We remark that some of our results are specific to this workload, and where that is the case we say so.

3 Cryptographic preliminaries

In this section we give the definitions, theorems and algorithms that the rest of this survey relies on. We present each construction in the form our artifact implements it, and we remark where this form differs from the textbook one, because several of our results depend precisely on those differences. In Section 3.1 we describe the security models and we define what hiding will mean. In Section 3.2 we present secret-sharing MPC and the three-party multiplication that needs no preprocessing. In Section 3.3 we describe garbled circuits and the three optimizations that determine their cost. In Section 3.4 we give the lattice background and the two encodings, BFV and BGV, that we implement. In Section 3.6 we show how a value can cross between a ciphertext and a sharing, and in Section 3.7 we briefly describe succinct proofs. For further information on the lattice material we point the reader to our previous report on the subject [47] and to the homomorphic encryption standard [1].

3.1 Security models

A protocol among n parties computes $f(x_1, \ldots, x_n)$ revealing nothing beyond the output. Security is defined by simulation: for every adversary in the real protocol there exists a simulator that, given only the corrupted parties' inputs and the output, produces an indistinguishable view. It means that whatever an adversary learns, it could have computed by itself from what it is entitled to know. Three axes determine the cost of achieving this.

Definition 3.1. An adversary is ***semi-honest*** when it follows the protocol and tries to learn from the messages it observes, and it is ***malicious*** when it deviates arbitrarily from the protocol. The ***corruption threshold*** t is the number of parties that the adversary controls; when $t < n/2$ we say that the protocol assumes an ***honest majority***, and when $t \le n-1$ we say that it tolerates a ***dishonest majority***. A protocol offers ***security with abort*** when the honest parties either output the correct value or detect the deviation and stop, and it offers ***guaranteed output delivery*** when they are required to finish.

The distance between these settings is not only theoretical. Semi-honest protocols require no consistency checks, and adding them is where most of the cost of active security is spent. An honest majority admits information-theoretic techniques and keeps public-key operations off the critical path, whereas a dishonest majority does not, which is why the SPDZ line [23, 22] needs cryptographic preprocessing. Our artifact is three-party with one corruption, which is the smallest honest-majority setting and also the setting of ABY3 [46] and Falcon [55]. Like them, we implement security with abort.

We also need a precise notion of what it means for a mask to hide a value, because one of our findings turns on it.

Definition 3.2. Let a be a secret in $\mathbb{Z}_{2^\ell}$ and let r be a mask drawn from a distribution over $\mathbb{Z}_{2^\ell}$. The masked value $c = a + r \bmod 2^\ell$ hides a ***perfectly*** when the distribution of c is independent of a, and it hides a ***statistically*** to within $2^{-\kappa}$ when, for every pair of secrets a, a', the distributions of $a + r$ and $a' + r$ are at total variation distance at most $2^{-\kappa}$.

Theorem 3.1. If r is uniform over the whole ring $\mathbb{Z}_{2^\ell}$, then $c = a + r$ is uniform and therefore hides a perfectly. If instead r is uniform over $[0, 2^m)$ with $m < \ell$ and $a \in [0, 2^m)$, then the posterior distribution of a given c is uniform over an interval of size $\min(c, 2^m) - \max(0, c - 2^m)$, which equals 1 when $c = 0$; for uniform a the mutual information is $I(a; c) \approx 1/(2 \ln 2) \approx 0.72$ bits.

The second case of Theorem 3.1 can be interpreted as the statement that a mask chosen only to prevent an overflow does not hide anything in particular. We give it as a theorem rather than as a remark because we shipped a truncation protocol that satisfied the first requirement and violated the second, and no correctness test could observe it; we describe the episode in Section 12.

3.2 Secret-sharing MPC

In this section we describe the sharing structures and the multiplication protocol that our MPC layer implements. We work over the ring $\mathbb{Z}_{2^k}$ rather than over a prime field, because machine arithmetic is already modular and because fixed-point truncation is more natural there.

Definition 3.3. An ***additive sharing*** of $x \in \mathbb{Z}_{2^k}$ among three parties is a triple (x_0, x_1, x_2) with $x_0 + x_1 + x_2 = x$, where party i holds x_i. A ***replicated sharing***, denominated $[x]$, gives party i the pair (x_i, x_{i+1}), with indices taken modulo 3, so that every additive component is held by exactly two parties.

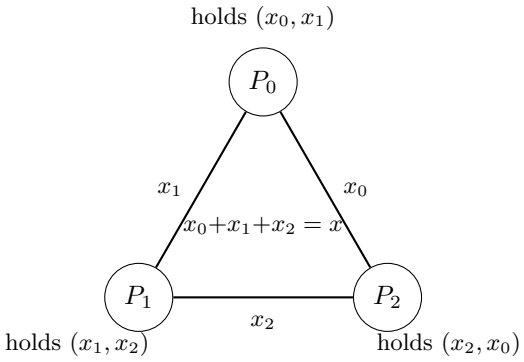


Figure 3.1: Replicated sharing over three parties. Each edge is the component shared by the two parties it connects, so any two parties together determine x and any single party learns nothing.

Figure 3.1 shows the structure. The redundancy costs storage and it buys two properties that we use repeatedly. Linear operations remain local, since each party applies them to both of its components. And every value that a party sends can be recomputed by some other party, which is what makes most deviations detectable by comparison; we return to this in the discussion of malicious security below.

Addition of two shared values is local. Multiplication is not, because the product xy expands into nine cross terms $x_i y_j$ that are distributed among the parties. The classical solution consumes a Beaver triple [4], that is, a preprocessed $([a], [b], [ab])$ with a, b random, which converts one multiplication into two openings. Under replicated sharing with three parties this preprocessing is unnecessary, and we consider this the single most important cost fact about the honest-majority setting.

Theorem 3.2 (ABY3 multiplication [46])**.** Let $[x]$ and $[y]$ be replicated sharings over $\mathbb{Z}_{2^k}$, and let $\alpha_0, \alpha_1, \alpha_2$ be a zero-sum triple, $\sum_i \alpha_i = 0$. If party i computes

$$z_i \;=\; x_i y_i \;+\; x_i y_{i+1} \;+\; x_{i+1} y_i \;+\; \alpha_i,$$

then $\sum_i z_i = xy$, and (z_0, z_1, z_2) is an additive sharing of xy that reveals nothing beyond it.

Proof. Summing the three expressions collects each of the nine products $x_i y_j$ exactly once, because the index pairs (i, i), $(i, i+1)$ and $(i+1, i)$ enumerate all of $\{0, 1, 2\}^2$ as i ranges over the three parties, and the α_i cancel by construction. Moreover, each z_i is masked by α_i, which depends on a key that party i does not hold in full, hence no party learns a cross term that it should not learn. □

The zero-sum randomness is obtained without communication. Party i holds PRF keys k_i and k_{i-1} shared with its two neighbours and computes $\alpha_i = F_{k_i}(\mathsf{id}) - F_{k_{i-1}}(\mathsf{id})$, where id is a counter that all parties advance identically; the sum telescopes to zero. Algorithm 3.1 states the whole protocol. We remark that its cost is one ring element per party and one round, with no preprocessing, and that this is why an honest-majority protocol can be cheaper online than a dishonest-majority one however efficient the latter's preprocessing becomes.

Crossing between arithmetic and Boolean domains. Comparisons, ReLU and argmax are not arithmetic circuits and they require the bits of a shared value. We denote by `a2b` the bit-decomposition protocol, which takes $[x]$ to sharings of its bits, and by `b2a` its inverse. Bit

Algorithm 3.1 Replicated-sharing multiplication

INPUT The shares (x_i, x_{i+1}) and (y_i, y_{i+1}) held by party i; the PRF keys k_i and k_{i-1}; the operation counter id. Indices are modulo 3.

OUTPUT The replicated sharing (z_i, z_{i+1}) of the product xy.

Compute $\alpha_i = F_{k_i}(\mathsf{id}) - F_{k_{i-1}}(\mathsf{id})$, which is local and zero-sum over the three parties.

Compute $z_i = x_i y_i + x_i y_{i+1} + x_{i+1} y_i + \alpha_i$, which is three of the nine cross terms.

Send z_i to party $i-1$ and receive z_{i+1} from party $i+1$, that is, one ring element in each direction and one round.

return (z_i, z_{i+1}).

decomposition is a carry-propagation problem, so its round complexity is decided by the adder: a ripple-carry adder needs depth $O(k)$, whereas a Kogge–Stone prefix adder [39] needs $O(\log k)$ at the price of more AND gates. Under replicated sharing the three additive components can be re-shared as Boolean values for free, since each one is already known to exactly the two parties that a replicated Boolean sharing requires, and therefore decomposition reduces to a three-input addition followed by a two-input one.

Malicious security by a multiplicative MAC. As we observed above, replicated sharing defends itself wherever a value is recomputable by another party. Auditing the semi-honest protocol leaves exactly one point that is not, namely the value z_i of Theorem 3.2, which depends on the pair (x_i, x_{i+1}) that no other party holds. Following Chida et al. [16] in the ring adaptation of Eerikson et al. [27], we carry every wire as a pair $([x], [r \cdot x])$ under a global MAC key r. Linear operations apply to both halves for free and a multiplication costs a second multiplication, so that a batched check at the end detects any deviation except with probability determined by the unused high bits of the ring. It means that the statistical security of the check is $128 -$ used bits, and consequently an actively secure execution cannot also narrow the ring wire to the width of its values: the check and the encoding compete for the same slack.

Fixed point and truncation. Weights are real numbers, so a value x represents $x/2^s$ for a scale s. A product of two such values carries the scale 2^{2s} and must be truncated by s bits. Truncation is the awkward primitive of this area. Local truncation of each share is cheap, introduces a one-bit error, and fails outright with small probability. Exact truncation opens a masked value and corrects the borrow, and it is therefore governed by Definition 3.2: the mask must be wide enough for correctness and uniform enough for privacy, which are not the same requirement.

3.3 Garbled circuits

In this section we describe Yao's construction [58] and the three optimizations that determine its modern cost. We use all three, and we remark that the third one is the reason why circuit design in this paradigm reduces to counting AND gates.

Definition 3.4. A ***garbling*** of a Boolean circuit assigns to each wire w two labels W_w^0, W_w^1, drawn uniformly from $\{0,1\}^\lambda$, and to each gate a table of ciphertexts such that the labels of the input values decrypt exactly the label of the correct output value. The ***garbler*** produces the tables, the ***evaluator*** obtains one label per wire, and by construction it cannot decide which value that label encodes.

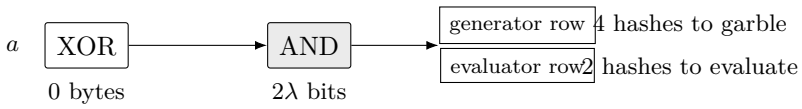


Figure 3.2: Cost of a garbled circuit under free-XOR and half-gates. XOR and NOT are free, so only the AND gates appear in the transcript.

Definition 3.5 (Point-and-permute)**.** Each label carries a random ***pointer bit***. The pointer bits of the input labels index the row of the gate table that must be decrypted, and therefore the evaluator performs one decryption instead of trying all the rows.

Definition 3.6 (Free-XOR [40])**.** Let $\Delta \in \{0,1\}^\lambda$ be a global offset with least significant bit 1, and let $W_w^1 = W_w^0 \oplus \Delta$ for every wire w. Then the output label of an XOR gate is the XOR of its input labels.

It means that XOR gates cost no ciphertexts and no hashing, and that negation is a relabelling: to compute NOT it is enough to flip which of the two labels denotes zero. Only AND gates consume bandwidth. We remark that the price of this optimization is a correlation among the labels, since all of them differ by the same Δ, and consequently the hash used inside a gate must remain secure when its inputs are correlated; a plain pseudorandom function is not sufficient.

Theorem 3.3 (Half-gates [59])**.** Under free-XOR, an AND gate can be garbled with two ciphertexts, by writing it as the XOR of a ***generator half-gate***, whose value depends on a bit that the garbler knows, and an ***evaluator half-gate***, whose value depends on a bit that the evaluator knows. Garbling one AND gate costs four hash evaluations and evaluating it costs two.

Figure 3.2 shows the resulting cost structure of a circuit. Together with Definition 3.6, Theorem 3.3 gives the accounting we use throughout: a circuit's communication is 2λ bits per AND gate and nothing else.

Three parties without oblivious transfer. In the two-party setting the evaluator must obtain the labels for its own input without revealing that input, which requires oblivious transfer. With three parties and an honest majority the oblivious transfer can be removed: two parties garble the same circuit from shared randomness and the third evaluates, so that input labels are delivered by the agreement of the two garblers, and consistency between the two garbled copies replaces the guarantee that oblivious transfer would provide. The protocol that results needs one round for the entire circuit. We remark that this is the structural reason why garbled circuits win under high latency, since the cost model is rounds $\times$ RTT + bytes/bandwidth and this paradigm minimizes the first term while secret sharing minimizes the second.

3.4 Lattices and homomorphic encryption

In this section we recall the lattice definitions that we need, we state the ring learning with errors problem, and we describe the two encodings, BFV and BGV, that our artifact implements. We keep the lattice material to what the cost discussion requires and we point the reader to our previous report [47] for the mathematical apparatus.

Definition 3.7. Let $B = [b_1, \dots, b_n]$ be a basis of linearly independent vectors in $\mathbb{R}^n$. The ***lattice*** generated by B is the set $\mathcal{L}(B) = \{Bx \mid x \in \mathbb{Z}^n\}$, and its ***fundamental domain*** is the parallelepiped $\mathcal{P}(B) = \{Bx \mid x \in [0,1)^n\}$. The determinant $\det(B)$ is independent of the choice of basis and corresponds geometrically to the inverse of the density of lattice points.

Figures 3.3 and 3.4 illustrate the two facts about lattices that the encryption schemes use. In Figure 3.3 a point $w \in \mathbb{R}^n$ decomposes uniquely as a lattice vector v plus an element t of the fundamental domain, and this decomposition can be interpreted as a modular reduction. In Figure 3.4 a ciphertext is a lattice point displaced by a small error e, so that decryption is the recovery of the nearest lattice point; it succeeds precisely while the error remains inside the shaded region, in which that point is unambiguous, and it fails for the ciphertext on the right, whose error has left it.

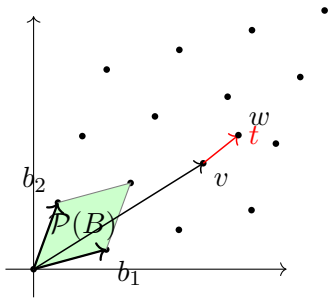


Figure 3.3: Reduction modulo $\mathcal{P}(B)$

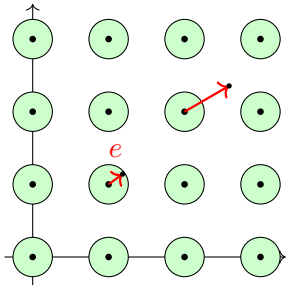


Figure 3.4: Noise budget of a ciphertext

Definition 3.8 (Ring learning with errors)**.** Let $R = \mathbb{Z}[X]/(X^n + 1)$ for a power of two n, and let $R_q = R/qR$. Given a secret $s \in R_q$, the RLWE distribution outputs pairs $(a, as + e)$ where a is uniform in R_q and e is drawn from a narrow error distribution. The ***decision*** problem is to distinguish such pairs from uniform ones, and the ***search*** problem is to recover s.

The hardness of Definition 3.8 reduces to worst-case lattice problems, and concrete parameters are obtained from lattice-reduction estimates [1, 2]. We remark that the security level depends jointly on n, on q and on the error distribution, and in particular that raising q at a fixed n *weakens* the scheme. It means that depth and security are coupled: a construction that needs less depth may use a smaller modulus and is therefore secure at a smaller ring dimension.

Definition 3.9 (BFV and BGV encodings)**.** Let t be the plaintext modulus and $\Delta = \lfloor q/t \rfloor$. A ciphertext c encrypts m under s with

$$\text{BFV [29]:}\quad c(s) = \Delta \cdot m + e, \qquad\qquad \text{BGV [8]:}\quad c(s) = m + t \cdot e.$$

Everything else follows from that single line. BFV places the message in the high-order bits and decrypts by scaling and rounding, so its tensor product carries a factor Δ^2 that must be scaled away; BGV places the message in the low-order bits and decrypts by reducing modulo t, so its noise grows multiplicatively and is controlled by *modulus switching*, that is, by dividing the ciphertext

modulus in order to divide the noise. We implement both, and we did so in order to answer a specific question by substitution rather than by argument: how much of the cost we measure belongs to RLWE itself and how much belongs to a particular encoding.

Noise, depth and bootstrapping. Every homomorphic operation enlarges the error of Figure 3.4, mildly for addition and sharply for multiplication. When the error leaves the region where the nearest lattice point is unambiguous, decryption fails, and therefore a parameter set supports a bounded multiplicative *depth*. Gentry's bootstrapping [31] evaluates the decryption circuit homomorphically and resets the noise, which makes unbounded depth possible and is the dominant cost of full homomorphism. It means that the practical question for a pipeline is not how fast a bootstrap is, but whether it occurs at all; in Section 6.4 we describe a pipeline whose depth is one by construction and which therefore performs none.

Packing. The ring R_q has n coefficients, so a single ciphertext carries many values. In *slot* packing the plaintext space is decomposed by the Chinese remainder theorem and operations act component-wise, and moving data between slots requires rotations. In *coefficient* packing we exploit the fact that polynomial multiplication is a convolution: if a vector is laid out forwards and a kernel is laid out *reversed*, then one ciphertext product places an inner product in a coefficient of the result, and with suitable strides it places an entire matrix–vector product or convolution there. This is the Gazelle [35] and Cheetah [32] line, and it is the reason why the cost of our convolutions does not depend on the image size.

Relinearization, and how to avoid it. A product of two ciphertexts decrypts under s^2 instead of s, and implementations normally *relinearize* back to s using an evaluation key. We remark that if the pipeline never multiplies again, then relinearization is pure overhead, since it is enough to decrypt under the squared key. Our construction does exactly that, and consequently evaluation keys disappear from the trusted setup and from the network. This simplification is available only because the depth is one.

Threshold decryption. When the secret key must not exist in a single place, it is shared and decryption becomes a protocol. We implement threshold decryption, and the key it decrypts under is generated distributively: no party holds it and no dealer produces it (Section 12). Both fresh and once-multiplied ciphertexts are covered, the latter without an evaluation key, for a reason the depth-1 design supplies and which we return to below.

3.5 A composability theorem, and what it costs

In this section we state what we can prove about composition, and we describe the two protocol changes that were required in order to be able to state it. We remark that the exercise was not bookkeeping: attempting the proof located a leak in Π_{PC} that had survived every correctness test, and we describe it in Section 12.

Definition 3.10. The ***arithmetic black box*** $\mathcal{F}_{\mathsf{ABB}}$ over $\mathbb{Z}_{2^k}$ is the ideal functionality that keeps a store of values indexed by handle and accepts the commands input, add, mul, lt, trunc_d and reveal. Only reveal produces output, and the functionality leaks to the adversary the sequence of commands together with the handles they name, and nothing else.

The leakage clause of Definition 3.10 is the one that carries weight, and it is why the control flow of a program matters. Our gadget library never branches on a secret, so its command sequence is a function of the public layer shapes alone; the interpreter, in contrast, implements if on a secret predicate by revealing that predicate, and therefore the statement below is about the library and not about the language.

Functionality $\mathcal{F}^{k}_{\mathsf{ABB}}$

The functionality keeps a partial map St from handles to $\mathbb{Z}_{2^k}$, initially empty, and interacts with parties P_0, P_1, P_2 and the adversary $\mathcal{S}$. Every command is ignored unless its input handles are defined and its output handle is not. On each command the functionality leaks the command name together with its handles to $\mathcal{S}$, and nothing else.

- $(\mathsf{input}, id, P_j)$: on this command from all parties and $x \in \mathbb{Z}_{2^k}$ from P_j, set $\mathsf{St}[id] \leftarrow x$.
- $(\mathsf{add}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow \mathsf{St}[id_1] + \mathsf{St}[id_2]$.
- $(\mathsf{mul}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow \mathsf{St}[id_1] \cdot \mathsf{St}[id_2]$.
- $(\mathsf{lt}, id_1, id_2, id_3)$: set $\mathsf{St}[id_3] \leftarrow 1$ if $\mathsf{St}[id_1] < \mathsf{St}[id_2]$ as signed k-bit integers, and 0 otherwise.
- $(\mathsf{trunc}_d, id_1, id_2)$: if $0 \le \mathsf{St}[id_1] < 2^{k-1}$ set $\mathsf{St}[id_2] \leftarrow \lfloor \mathsf{St}[id_1]/2^d \rfloor$; otherwise set $\mathsf{St}[id_2] \leftarrow \perp$.
- (reveal, id, R): send $\mathsf{St}[id]$ to every party in R.
- (abort) from $\mathcal{S}$: send $\perp$ to every honest party and halt. *Present only in the malicious variant* $\mathcal{F}^{k,\mathrm{abort}}_{\mathsf{ABB}}$.

Figure 3.5: The arithmetic black box our MPC layer realizes

Figure 3.5 makes two design decisions explicit, and both of them are load-bearing.

The first is the ***leakage clause***. The functionality tells the adversary which commands were issued and on which handles, and therefore a protocol realizing it is allowed to have a public control flow and nothing more. It means that a program whose sequence of operations depends on a secret cannot be simulated, which is why Corolary 3.2 is a statement about our gadget library, whose command sequence is a function of the layer shapes, and not about our language, whose if on a secret predicate reveals that predicate.

The second is the guard on trunc_d. We could have written truncation as a total function on $\mathbb{Z}_{2^k}$, and then no protocol of ours would realize it, because our truncation is correct only for a non-negative argument below half the ring. Writing the precondition into the functionality instead is the honest move: the guard is exactly the bound that Section 3.2 obtains by gating before shifting, and a caller that violates it receives $\perp$ rather than a wrong answer. We remark that this is the general shape of the "no free lunch" in a UC statement — either the protocol is strengthened to meet the functionality, or the functionality is weakened to state what the protocol does, and the second is only legitimate when the weakening is written down.

Figure 3.6 recalls what has to be established. The environment $\mathcal{Z}$ chooses the inputs, sees the outputs, and talks to the adversary throughout; the protocol realizes the functionality when for every $\mathcal{A}$ there is a simulator $\mathcal{S}$ such that no $\mathcal{Z}$ distinguishes its interaction with the real protocol from its interaction with $\mathcal{F}^{k}_{\mathsf{ABB}}$. We remark that $\mathcal{Z}$ is what makes the statement composable and also what makes it demanding: since $\mathcal{Z}$ may run other sessions and feed their results back, a simulator is not allowed to rewind, and any randomness that a protocol reuses across sessions is randomness that $\mathcal{Z}$ can arrange to collide.

Theorem 3.4 (Simulator)**.** Let $\mathcal{A}$ be a static semi-honest adversary corrupting P_c. There is a straight-line simulator $\mathcal{S}$ that, given the leakage of Figure 3.5 and the corrupted party's inputs and outputs, produces a view computationally indistinguishable from the real one.

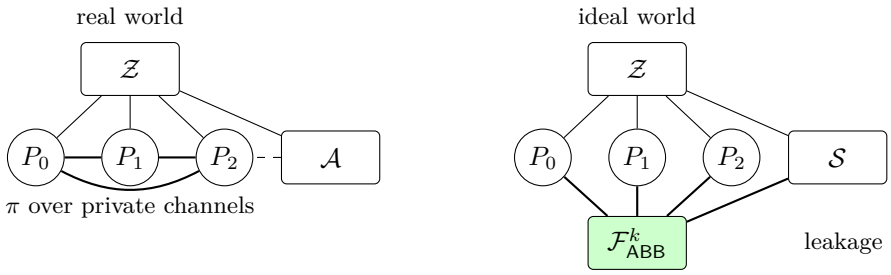


Figure 3.6: UC realization: no $\mathcal{Z}$ distinguishes the two

Proof. We construct $\mathcal{S}$ command by command; in every case it holds the corrupted party's two replicated components and must produce the messages that party receives.

For input and for the linear commands there are no messages, since a replicated sharing is created locally from the dealer's value and addition acts on components.

For mul, the real protocol delivers to P_c the single element $z_{c+1} = x_{c+1}y_{c+1}+x_{c+1}y_{c+2}+x_{c+2}y_{c+1}+\alpha_{c+1}$. The term α_{c+1} is $F_{k_{c+1}}(\cdot) - F_{k_c}(\cdot)$, and P_c holds k_c but not k_{c+1}, so by pseudorandomness of F the element is computationally indistinguishable from uniform. $\mathcal{S}$ samples it uniformly. This is the step that needs the keys to be fresh per session: were the same (k, ctr) to recur in a second session that $\mathcal{Z}$ schedules, the two elements would differ by a value $\mathcal{Z}$ knows, and no uniform sample reproduces that.

For lt and trunc_d, the messages are those of Π_{WA}, Π_{PC}, Π_{SS} and Π_{Trunc}, and each of their openings is covered by Lemma 3.1: $\mathcal{S}$ samples the opened value from the distribution the lemma names — uniform on the ring, a uniform bit, or uniform on $\mathbb{Z}_p^\star$ conditioned on the predicate the functionality has already given it — and derives the remaining messages from the corrupted party's components exactly as the protocol does.

For reveal, the functionality supplies the value and $\mathcal{S}$ sends the missing component consistent with it.

Indistinguishability follows by a standard hybrid over the PRF calls, of which there are polynomially many, replacing each in turn by a uniform value. No step rewinds $\mathcal{Z}$ or $\mathcal{A}$. □

Corolary 3.1. With the MAC layer of Chida et al. [16] in the ring adaptation of Eerikson et al. [27], the same protocol realizes $\mathcal{F}^{k,\mathrm{abort}}_{\mathsf{ABB}}$ against a static malicious adversary, provided every consistency check is deferred to one point whose outcome depends only on whether the adversary deviated.

The proviso in Corolary 3.1 is not a formality. If a check can fail as a function of a secret, then the abort itself is an output channel that $\mathcal{F}^{k,\mathrm{abort}}_{\mathsf{ABB}}$ does not have, and no simulator reproduces it: the simulator learns from the functionality whether the adversary cheated, and never learns the secret that a data-dependent check would consult. Our own first attempt at this layer had exactly that defect, and we describe it in Section 11; Brüggemann and Schneider [9] exploit the same shape against Trident [13], Fantastic Four [21] and Quad, and identify a gap of this kind in the proof of SWIFT [41], which is evidence that the condition is easy to violate while believing one has batched a check for efficiency alone.

On the preprocessing. The masks of Lemma 3.1 are produced by the pseudorandom secret-sharing of Cramer, Damgård and Ishai [20] and by doubly-authenticated bits in the sense of Rotaru and Wood [50], improved by the edaBits of Escudero et al. [28]. We remark that the functionality

they realize is a preprocessing functionality and not $\mathcal{F}_{\mathsf{ABB}}$, so the composition is two-layered: the preprocessing realizes $\mathcal{F}_{\mathrm{Prep}}$, and the online phase realizes $\mathcal{F}_{\mathsf{ABB}}$ in the $\mathcal{F}_{\mathrm{Prep}}$-hybrid model. It means that our pool of Section 9.3 is not merely an optimization but the boundary between the two layers of the proof, and that the miss counter which refuses to let an online phase generate its own preprocessing is enforcing that boundary rather than only measuring it.

Lemma 3.1 (Opening audit)**.** In the protocols of this artifact, every value opened to the parties is either the output of a reveal command or is masked by randomness that is uniform on the domain of the opening and used only once. Consequently each opening is simulatable from the corresponding command's output.

Proof. By enumeration; the sites are finitely many and Table 1 lists them. The three interesting cases are as follows. In Π_{WA} the opened value is $a + x$ for a preprocessed x that is uniform on the whole ring, so the opening is uniform. In Π_{Trunc} the same holds, which is the reason the mask must cover the ring and the overflow be corrected rather than avoided. In Π_{PC} the opened value is $m \cdot \prod_i c_i$; the c_i carry no randomness of their own, hence the mask m must be a uniform unit *and must remain secret*, which is what forces the zero test on m to open $m \cdot s$ rather than m. □

Protocol	Opened value	Mask	Simulatable
Π_{PC}	$m \cdot \prod_i c_i$	m uniform unit, secret	yes†
Π_{WA}	components of $a + x$	x uniform on $\mathbb{Z}_{2^\ell}$	yes
Π_{SS}	$b \oplus c$	c a fresh uniform bit	yes
Π_{Trunc}	$a + r$	r uniform on $\mathbb{Z}_{2^\ell}$	yes†
mask generation	$\delta \oplus r$	r a fresh uniform bit	yes
unit sampling	$m \cdot s$	s uniform on $\mathbb{Z}_p$	yes†
reveal	the command's output	—	by definition
FHE → MPC	$m + \sum_i r_i$	r_i uniform on $\mathbb{Z}_{2^k}$	yes
a2b	nothing	—	by definition

Table 1: Every opening, and why it can be simulated. † three of the nine required a protocol change before the entry could be written; see Section 12.

Theorem 3.5. Assume private authenticated point-to-point channels, a pseudorandom function, and a setup that delivers pairwise PRF keys freshly per session. Then the protocol of Section 3.2 UC-realizes $\mathcal{F}_{\mathsf{ABB}}$ over $\mathbb{Z}_{2^k}$ against a static adversary corrupting at most one of three parties, semi-honestly; and with the MAC layer of Chida et al. [16] in the ring adaptation of Eerikson et al. [27], against a static malicious adversary, with abort.

We do not claim a machine-checked proof, and we set out the argument and its hypotheses so that the gaps are visible. Simulation is straight-line, since the simulator for a semi-honest minority samples the corrupt party's view directly: linear operations are local, the multiplication of Theorem 3.2 sends one element masked by an α_i the corrupt party cannot compute, and every opening is handled by Lemma 3.1. No rewinding appears, which is what distinguishes a UC argument from a standalone one. Keys are drawn afresh per session, so a one-time mask indexed by an operation counter is never reused across the concurrent sessions that the UC environment may schedule; this is a property of our transport and we state it as a hypothesis because it is not enforced by a type. For the malicious case the abort must be simulatable, and it is: the checks are deferred to a single point whose outcome depends on whether the adversary deviated and not on any secret. We remark that this is exactly the property whose absence Brüggemann and Schneider [9] exploit

against Trident, Fantastic Four and Quad, and whose specification gap they identify in SWIFT; deferring a check hides *when* an abort happens and not *whether* it happens, and only the latter is what security requires.

Corolary 3.2. Any network assembled from the gadget library, whose control flow is determined by public layer shapes, UC-realizes the functionality that reveals only its declared output.

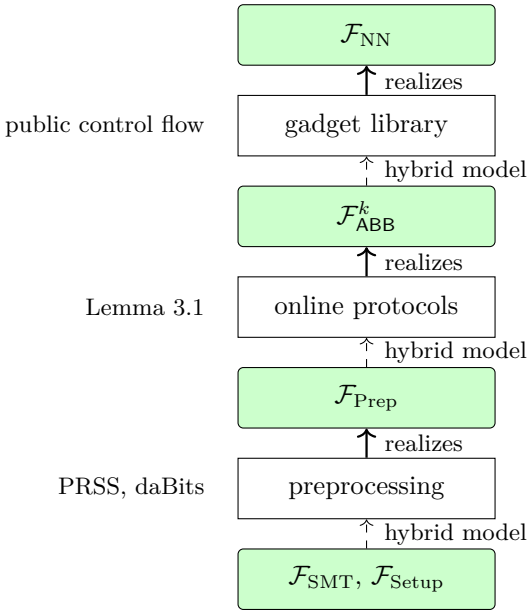


Figure 3.7: The layers the composition theorem is applied through

Figure 3.7 shows the shape of the argument, and we draw it because the layering is what makes each step small enough to check. Reading upwards, the preprocessing realizes $\mathcal{F}_{\text{Prep}}$ from private channels and the key setup; the online protocols realize $\mathcal{F}_{\mathsf{ABB}}$ in the $\mathcal{F}_{\text{Prep}}$-hybrid model, which is the step Lemma 3.1 and Theorem 3.4 establish; and the gadget library realizes $\mathcal{F}_{\text{NN}}$ in the $\mathcal{F}_{\mathsf{ABB}}$-hybrid model, which is Corolary 3.2 and needs only that the control flow be public. It means that the boundary between the second and third layers is the pool of Section 9.3: a warm server has run the bottom layer in advance, and the miss counter that refuses to let an online phase generate its own preprocessing is enforcing the layer boundary and not merely reporting a latency.

We remark that $\mathcal{F}_{\text{NN}}$ is the weakest of the three to specify and the easiest to get wrong, because it is where the application decides what may be public. Ours reveals the predicted class and the network's architecture, and nothing else: not the logits, not the activations, and not the input. Writing it down is what makes "the pipeline reveals only the class index" a checkable statement rather than a description of the code.

What the theorem does not cover, and what it would cost. The hybrid pipeline is outside $\mathcal{F}_{\mathsf{ABB}}$, because $\mathcal{F}_{\mathsf{ABB}}$ has no ciphertexts. Extending the statement to it needs three things, and we have two. The conversion of Section 3.6 opens $m + \sum_i r_i$ with each r_i uniform on the plaintext space, so it satisfies Lemma 3.1. Against a malicious adversary an input ciphertext must also be shown well-formed with bounded noise, because replication cannot detect a malformed one — both honest co-holders compute on it faithfully and agree — and that proof is implemented.

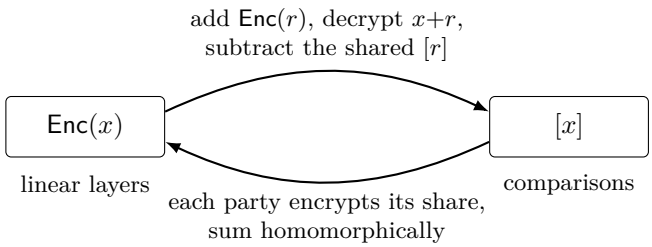


Figure 3.8: Scheme switching. Both directions are linear, so a hybrid pipeline can alternate between them without consuming multiplicative depth.

Key generation was the last part of this argument to rest on a setup assumption, and it no longer does: the key is sampled distributively, so the hybrid realizes its functionality without a dealer rather than in a model with one. The construction follows the line Damgård and Nielsen [22] established for an arithmetic black box from threshold homomorphic encryption and which Asharov et al. [3] carry to threshold FHE, with one adaptation that the replicated setting forces and one dividend that it pays; both are described in Section 12. The adaptation is that components are ternary rather than uniform, because independent sampling cannot deliver a small key and uniform shares at once, and the resulting leak is one the standard assumption already absorbs. The dividend is that a product ciphertext needs no evaluation key at all, since replicated shares yield a sharing of s^2 with no communication.

What remains is not a setup assumption but a parameter one, and we state it in Section 12: the boundary proof's soundness slack is subtracted from the smudging budget bit for bit, which leaves a smaller λ than an honest-input analysis reports.

3.6 Crossing between the paradigms

A ciphertext and a secret sharing are two representations of the same hidden value, and the possibility of converting between them is what makes a hybrid protocol possible. In this section we describe the two conversions, which are illustrated in Figure 3.8. We remark that both are linear, and consequently neither consumes multiplicative depth, which is the property that Section 6.4 depends on.

To pass from a ciphertext to a sharing, we add a fresh random mask homomorphically, we decrypt the masked ciphertext, and we let the mask itself be secret-shared. The plaintext that appears is uniform by Theorem 3.1, and subtracting the shared mask leaves a sharing of the original value. To pass from a sharing to a ciphertext, each party encrypts its own share and the ciphertexts are added homomorphically, which is correct because the sharing is additive.

3.7 Succinct proofs

A zero-knowledge argument allows a prover to convince a verifier that it knows a witness satisfying a public relation, revealing nothing else, and it is *succinct* when the proof size and the verification time are sublinear in the computation being proved. We remark that these systems answer a different question from the one that MPC and homomorphic encryption answer: they establish the *integrity* of a computation performed by somebody else, and not the privacy of a computation performed jointly. They enter this survey because a deployment of private inference may need both properties at once. Our artifact targets Binius64 [25], which operates over towers of binary fields so that operations on 64-bit words are expressed without bit-blasting them.

Paradigm	Secret from	Expensive in	Cheap in	Rounds
Secret-sharing MPC	computing parties	**rounds**, bandwidth	local arithmetic	O(depth)
Garbled circuits	computing parties	**bandwidth**	rounds	$O(1)$
FHE	computing server	**compute**, ct. size	rounds (none)	none
ZK proofs	the verifier	**prover compute**	verifier, proof size	none

Table 2: The paradigms organized by cost structure rather than security property. Three trade-offs follow directly, and drive most of this paper.

4 Four paradigms, four cost structures

In this section we organize the four paradigms. They are usually taxonomized by the security property that they provide, but for a practitioner who has to choose among them we consider that the more useful axis is *what each one is expensive in*, and this is the axis of Table 2. Three trade-offs follow from it directly, and they drive most of the remainder of this survey.

Secret sharing and garbled circuits trade rounds against bytes. This is the classical trade between the constructions of Yao [58] and of GMW, and neither of them dominates the other. The winner depends on the round-trip time and on the bandwidth, and therefore the location of the boundary is an empirical question. We measure it in Section 9.3, where we also report the three answers that we published before the current one.

Homomorphic encryption and secret sharing trade compute against communication. Homomorphic encryption sends much data once and then computes hard, whereas secret sharing sends little data many times. On a fast local network secret sharing usually wins, and on a thin pipe the balance moves. We remark that this framing omits *preprocessing*, and in Section 9.1 we show that the omission is precisely what makes most published crossovers between the two paradigms fragile.

Succinct proofs are orthogonal, and conflating them with the others is a category error. Secret sharing and homomorphic encryption keep data secret *from the parties that compute on it*. A succinct proof establishes that a computation was performed correctly while hiding the witness *from a verifier*, and the prover holds everything in the clear. It means that a deployment which needs both confidentiality and integrity pays for both, and that a proving time and a protocol latency do not belong in the same column. For this reason we keep them in separate tables, and our benchmark prints the caveat next to its numbers so that they cannot travel without it.

4.1 The hybrid, and why it exists

Inference in a neural network alternates linear layers with nonlinearities. Homomorphic encryption is good at the former and bad at the latter, since a comparison is not a low-degree polynomial; secret sharing is the reverse, since a comparison costs a bit decomposition whereas a linear layer costs one round independently of its width.

Consequently the natural design is to evaluate the linear layers under homomorphic encryption and the nonlinearities under secret sharing, with a *scheme switch* between them in the sense of Section 3.6. This is the design that Gazelle [35], Delphi [45] and Cheetah [32] instantiate. We implement it and we add two elements, which we describe in Section 6.4: the weights are encrypted

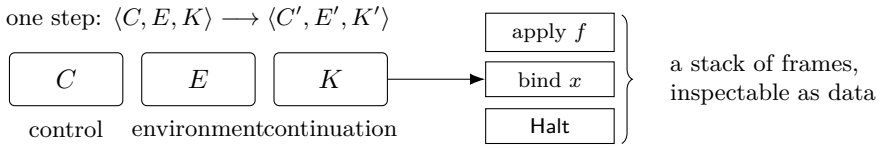


Figure 5.9: One step of the abstract machine. Because the continuation is data and not the host stack, it can be inspected, serialized, and used to batch independent protocol calls.

as well, and the switch is used as a *noise refresh*, which removes bootstrapping from deep networks entirely.

5 The Baru artifact

5.1 Baru is a CEK machine

In this section we describe the abstract machine at the centre of our artifact, and we argue that this choice is what makes the comparisons of this survey possible. Our artifact is not a circuit compiler with a front end. It is an abstract machine for a Lisp-like language, and the cryptography is placed behind a single interface that the machine calls.

Definition 5.1. A ***CEK machine*** [30] evaluates a program by rewriting a triple $\langle C, E, K\rangle$, denominated a ***configuration***, where C is the expression under evaluation, E is an environment that maps variables to values, and K is a ***continuation***, that is, a representation as data of what remains to be done once the value of C is known.

Figure 5.9 shows one step of the machine. Evaluation is a loop over a step function: in order to evaluate an application $(f\ a)$, the machine pushes onto K the instruction "evaluate the argument and then apply f" and moves to f; when a value is finally produced, the machine pops the topmost frame of K and proceeds. It means that no recursion takes place on the stack of the host language, and consequently a deep program consumes heap rather than stack, and the entire state of the machine is an ordinary value.

Why an abstract machine and not a circuit. The dominant design for secure computation compiles a program into a circuit and then evaluates that circuit under one protocol, as in ABY [24], HyCC [10], CirC [48] and Silph [14]. We consider that this is the correct design when the protocol is fixed, and the incorrect one for a survey, because a circuit has already made the choices of the protocol. A comparison compiled for garbling is a Boolean circuit, and the same comparison compiled for arithmetic sharing is not the same object. It means that comparing two paradigms would reduce to comparing two compilations, and any difference between the two compilations would contaminate the measurement.

An interpreter, in contrast, keeps the program fixed and changes what a primitive *means*. Our evaluator manipulates secret values only through opaque handles and a fixed set of operations, which is the Arithmetic Black Box of the SPDZ line [22, 23]. Therefore the expression `(> z 0)` is one call to `lt`, and whether that call becomes a bit decomposition, a garbled comparator or a `wrap3` sign extraction is a decision taken by a backend at run time. In other words, the same source text, the same syntax tree and the same sequence of calls to $\mathcal{F}_{\mathsf{ABB}}$ reach every paradigm, and consequently a

difference that we measure between two paradigms is attributable to the protocol and not to the program.

What the continuation buys. We use three consequences of Definition 5.1 in this work.

- **Non-local control is data.** Condition handlers are frames on K, so that a `guard` installs one and leaving its body pops it. Therefore error handling inside a private computation requires no exceptions of the host language, and it cannot accidentally unwind past a point at which a protocol must synchronize.
- **The abstraction admits batching, although the machine does not perform it.** A backend primitive receives a *vector* of operands, so an entire layer of `lt` calls can be one protocol round rather than thousands. We are precise about where that batching comes from, because it is not from the continuation: our evaluator issues primitives one at a time, and the parallelism is supplied either by a gadget that calls the batched primitive explicitly, or by the compiler of Section 9, which recovers it automatically by levelling a dependence graph. What the abstract machine contributes is the seam that lets both routes exist behind one program. We remark that this distinction matters more than it appears to, since almost every cost result in this survey depends on the difference between a cost paid per element and a cost paid per call.
- **The state is serializable.** A configuration is an ordinary value, so an execution can be checkpointed, replayed, or, as we use it here, run three times in lockstep by three parties whose operation identifiers are required to agree. We derive protocol randomness from a counter that every party advances identically, and this is sound only because the three machines take the same steps in the same order.

The cost of this choice. Interpretation is slower than compilation, and we make no claim about absolute performance (Section 12). An interpreter also cannot perform the whole-circuit optimizations that a compiler would find, and therefore when a compiler-based system reports fewer AND gates for the same function, that difference is real and it is not a difference between protocols. What our design buys is not speed but *commensurability*, and in Section 11 we record cases in which commensurability is what caught an error that the numbers alone did not reveal.

5.2 One program, many backends

In this section we describe the interface through which the machine of Section 5.1 reaches the cryptography. As we said above, our language manipulates secret values only through opaque handles and a fixed set of primitive operations, which is the Arithmetic Black Box abstraction of the SPDZ line [22, 23]: an ideal functionality $\mathcal{F}_{\mathsf{ABB}}$ that stores values by handle and exposes `add`, `mul`, `lt`, `share_public` and `reveal`. A backend implements those primitives, and the evaluator never inspects a representation.

```
; private ReLU on one neuron -- every value is secret-shared
(let ((w :sint8 -1) (x :sint8 3) (b :sint8 0))
  (let ((z (+ (* w x) b)))
    (reconstruct (* z (> z 0)))))          ; => 0

; a value computed under MPC crosses into FHE without being revealed
(let ((keys (fhe-keygen)) (x (share 6 32)) (y (share -7 32)))
  (fhe-dec keys (share->fhe keys (* x y))))   ; => -42
```

Layer	Contents
MPC	ABY3 [46] replicated shares over $\mathbb{Z}_{2^{128}}$; **triple-free** multiplication from pairwise PRFs; bit decomposition (carry-save $3 \to 2$ plus Kogge–Stone [39]); batched primitives; **malicious security with abort** (≤ 1 of 3 corrupt), following Chida et al. [16] in the ring adaptation of Eerikson et al. [27]; in-process and TCP transports
Garbled	Half-gates [59], free-XOR [40], fixed-key AES; comparator and fused ReLU $\circ$ rescale circuits; OT-free label delivery in the three-party honest-majority setting
Falcon	Π_{PC} private compare over $\mathbb{Z}_p$, Π_{WA} `wrap3`, Π_{DReLU}, Π_{SS}, Π_{Maxpool}, exact truncation—a reproduction of Falcon [55] inside the same harness
FHE	BFV [29] and BGV [8] (depth-one, no evaluation keys), TFHE-rs [53, 17], an exact real-arithmetic backend; multi-modular NTT with Shoup butterflies; CRT slot packing; Cheetah-style [32] convolution packing
Hybrid	MPC$\leftrightarrow$FHE switching both directions; re-encryption as a bootstrapping alternative; replicated threshold-key switch [3]
ZKP	Expression-to-circuit compiler; real Binius64 [25] prover and verifier
Compiler	Backend-aware circuit IR, rewrite rules, cost models, protocol registry (dealer / Shamir / Semi2k [36] / LowGear [37]), per-operation protocol assignment
Primitives	Chou–Orlandi [18] and IKNP [33] OT, DH-PSI

Table 3: What the artifact implements. Every row is exercised by at least one benchmark, and every engine attests its security level (Section 8.4).

The consequence for a survey is immediate. *The same source text* is the workload for every paradigm, and therefore a comparison between paradigms cannot be confounded by two implementations that compute subtly different functions. We remark that this is not a hypothetical concern, since in Section 11 we record a case in which it caught exactly that.

5.3 Composability

Every gadget of our library, namely `relu`, `max2`, `maxpool2d`, `argmax`, `dense` and `conv2d`, is a pure composition of $\mathcal{F}_{\mathsf{ABB}}$ primitives: no gadget opens a value and none branches on a secret, so the sequence of primitive calls a gadget makes is a function of the public layer shapes. Consequently the composition theorem of the universal composability framework [11] carries security from the primitives to any network assembled from the gadgets, and a search over architectures requires no new security argument for each architecture. It means that the research question becomes which architecture is *fastest* under the cost model, rather than whether a given one is secure.

We remark that the hypothesis of that sentence is stronger than it looks, and we state it properly in Section 3.5. The primitives must *UC-realize* $\mathcal{F}_{\mathsf{ABB}}$, which is not implied by their being secure in a standalone sense, and every value the primitive protocols open internally must be simulatable. Two of our own security failures were failures of precisely that hypothesis rather than of composition, so we treat the gadget-level claim as conditional and prove the condition separately.

5.4 What is implemented

Table 3 summarizes what the artifact contains. Unless we note otherwise, all the figures that follow were obtained on the same machine, an Apple M1 Pro with 8 cores, in a release build.

Workload	bytes		int. calls	wall clock (WAN, 25 ms)
	equal width	packed base		
Comparisons, n=32, 32-bit	**2.63**×	**9.70**×	**1.06**×	**1.26**×
Dot product, n=64, 32-bit	2.44×	2.85×	2.00×	2.26×

Table 4: Semi-honest ABY3 versus malicious-with-abort, same workload, same transport. Active security costs bytes, not rounds. *Equal width* charges both legs a 128-bit wire; *packed base* lets the semi-honest leg pack to its values' own width, which the verified leg cannot do because its statistical security is $128 - \mathit{used_bits}$. The two differ by 3.7× on the workload that matters for inference.

6 The paradigms, surveyed and measured

In this section we survey and measure each paradigm in turn, in the order in which Section 3 introduced them.

6.1 Secret-sharing MPC

Our MPC layer follows ABY3 [46], so that $x = s_0 + s_1 + s_2 \bmod 2^{128}$ and party i holds the replicated pair (s_i, s_{i+1}) of Definition 3.3. Addition and multiplication by a scalar are local, and multiplication is the triple-free protocol of Theorem 3.2, in which each party sends one ring element to one neighbour and the correlated randomness is regenerated locally from a single pairwise exchange of PRF keys. There is no dealer and there is no preprocessing phase.

We remark that this layer taught us the same engineering lesson on four separate occasions, and that the lesson generalizes beyond our artifact.

Finding. The arithmetic was never the bottleneck; the round structure was. Each time a segment-level measurement said "MPC is slow", a per-phase breakdown said "one unbatched loop inside it is slow".

A batched form costs *the same number of rounds for any batch width*. When we adopted such forms throughout, the MPC segment of our convolutional network moved from 40.8 s to 6.1 s, that is, by a factor of 6.7×, and the number of messages fell from 29,211 to 5,591. It means that a published MPC cost is highly sensitive to the batching discipline of its implementation, and consequently that comparing the round counts of two systems, without first verifying that both of them batch, compares implementations and not protocols.

The cost of active security. We measured the malicious protocol of Section 3.2 against the semi-honest one on the same workload and the same transport, and we give the result in Table 4.

The row for comparisons is the one that matters for inference, since every ReLU, every max-pool and every argmax reduces to a comparison.

Finding. Active security costs bytes, not rounds: the MAC doubles what is sent but batches into the same round.

The dot-product row is a small-workload artifact—the semi-honest leg needs only four interactive calls, so a fixed pair-pool cost amortizes against nothing—and we report it rather than hide it.

A second result generalizes, **with a correction we owe to Section 9.3**. The statistical slack that ring-based verification normally charges as a widening to $\mathbb{Z}_{2^{k+s}}$ [19] is already *present* when the implementation shares over a ring much wider than its values: Q16.16 values in a 128-bit ring leave **59 bits** of statistical security.

We previously added that this came at *no widening cost*. That is false, and packing the semi-honest paths is what exposed it: the 128-bit ring had been costing 3.1–5.6× in bytes on every one of them, which is exactly what narrowing to the values' own width recovered. The verified path cannot recover it—its security is $128 - \mathit{used_bits}$, so narrowing to the value width sets λ to zero. The slack was never free; it was invisible, because nothing was packed and so nothing revealed its price. The 2.63× above is therefore a fair ratio at equal width that *understates* the cost of active security against a packed baseline. Measured: 9.70× on the comparison workload—the semi-honest leg packs from 234,036 to 63,416 B while the verified leg is unchanged at 615,446. The ratio of ratios, 3.69×, falls inside the packing factors we measured on comparable paths, as predicted.

6.2 Garbled circuits

As we described in Section 3.3, in the three-party honest-majority setting the input labels are delivered by the co-garbler and no oblivious transfer is required. Since oblivious transfer is the expensive part of two-party garbling, this changes the economics substantially, and it is the reason why garbling is competitive in this setting at all.

We measured the two routes head to head over the identical transport, on 1,352 comparisons of 20-bit values. Bit decomposition needs 9 rounds and 1.29 MB, whereas the garbled circuit needs 1 round and 1.69 MB. It means that **bit decomposition is the byte-cheaper of the two**, by a factor of 1.3×, and that the advantage of garbling lies entirely in the rounds: the wall clock favours it by 18.8× on a local network and by 4.0× on a wide-area network with 80 ms of round-trip time and 100 Mbit s^{-1} of bandwidth, the ratio shrinking as bandwidth rather than latency becomes the binding term.

This result reverses a figure that we published. The same experiment previously reported bit decomposition at 5.94 MB, which made garbling appear 3.5× cheaper in bytes *as well as* faster. That difference of 4.6× was serialisation padding: a 20-bit ring share travelled inside 128 bits, and bit decomposition is almost entirely ring traffic whereas garbled tables are already compact. It means that the overhead fell on one side of the comparison only. Once the encoding was corrected, as we describe in Section 9.3, the ordering in bytes inverts and only the advantage in rounds survives. We remark that both routes remained bit-exact against the plaintext reference throughout, and consequently no correctness test could have revealed the problem.

A correction to another of our claims. The figures given above are for the *garbling primitive in isolation*. The deployable three-party protocol is smaller, once we include the $Y2A$ return leg that converts the Boolean output back into an arithmetic share: it is 7.7× on a local network, falling to 1.8× at 80 ms and 100 Mbit s^{-1}, and between 3.9× and 2.4× for the fused ReLU $\circ$ rescale. We record both figures, because the distance between them is exactly what is lost when a number is cited.

An earlier version of this paragraph quoted between 1.4× and 1.7× for the deployable path. That figure was taken not from this experiment but from the *MPC segment* of our convolutional network, which is a different benchmark on a different workload, and it was presented as though it corrected this one. That segment measures 6.1× today. The old figure was correct when it was written, and it became stale for a reason that we record in Section 11: an unbatched scheme-switch import was then 94% of the MPC segment, which loaded both routes equally and therefore compressed the ratio toward one. We consider that both corrections illustrate a single rule, namely that one should quote the deployable protocol, on the workload about which the claim is being made, and name the benchmark that produced the number.

6.3 Fully homomorphic encryption

We use the cheapest homomorphic primitive that exists, namely a single ciphertext-by-ciphertext multiplication, with no relinearization keys, no evaluation keys and no bootstrapping, decrypting the resulting degree-two product directly under s^2 as we described in Section 3.4. Our parameters for BFV are $n = 4096$, $q = 2^{100}$ and $t = 2^{20}$, which lie inside the ceiling of approximately 109 bits on $\log q$ for 128-bit security at that ring dimension [1].

We consider that two results about packing are worth surveying.

- **Reversed-coefficient packing** turns an entire matrix–vector product into one multiplication. We find that it is *already optimal for dense layers*, since a product of degree n admits at most n/in separable outputs, and therefore $\lceil \text{out} \cdot \text{in}/n \rceil$ multiplications is the limit. The SIMD packing of Gazelle [35] matches this throughput, but it requires rotations and Galois keys, and consequently it forfeits the property of needing no evaluation keys.
- **Convolution packing** in the manner of Cheetah [32] turns an entire output channel into one multiplication, independently of the size of the image. We measured the two layouts against each other at 28×28, and 1,352 multiplications become **two**, while the time spent in homomorphic evaluation falls from 77.5 s to 264 ms.

Finding. BGV is indistinguishable from BFV at identical parameters (repeated runs put the gap at +0.5%, +0.5%, −0.1%). Where the message sits in the ciphertext, high bits or low, is not where the cost is.

On the arithmetic side, we use a multi-modular number-theoretic transform, in which NTT-friendly primes are employed as a computational device and the result is reconstructed by the Chinese remainder theorem, because a power-of-two modulus q admits no direct transform. After we moved its butterfly to Shoup-preconditioned multiplication, it became 1.8× faster than NTL together with GMP [52] at every size that we measured. We remark that we retain NTL as a *differential oracle* rather than as a dependency, and that this is what made it safe to rewrite the hottest code of the repository.

6.4 The hybrid and the no-bootstrap result

In this section we describe our hybrid construction and the consequence that we consider its main result. The central observation is that the scheme switch which we perform in order to evaluate a ReLU *also refreshes the ciphertext*. When we switch back in the direction of Section 3.6, each party encrypts its share component afresh and the evaluator adds the ciphertexts, and a sum of fresh encryptions is itself a fresh encryption. It means that the noise budget of Figure 3.4 is reset at every boundary between layers. Consequently a network of arbitrary depth runs with **zero bootstraps**, at a cost that is linear in the depth, on an engine that supports only a *single* multiplication. We give the measurements in Table 5.

We remark that our construction also encrypts the **weights**, and that this is a stronger privacy model than the one of Gazelle, Delphi and Cheetah, in which the server holds the model in plaintext. The exactness column of Table 5 is the evidence for the claim about noise, since accumulated noise would cause the outputs at large depth to diverge from the plaintext reference, and they do not.

6.5 Zero-knowledge proofs

In this section we measure the succinct-proof paradigm of Section 3.7. The statement that we prove is a quantized neuron in the Q16.16 format, namely $\text{ReLU}((\sum_i w_i x_i) \gg 16 + b)$, with public weights

depth	exact?	total	fhe	switch	mpc	our boots.	levelled-competitor boots.
2	✓	145 ms	20	59	54	**0**	0
8	✓	539 ms	81	239	210	**0**	3
32	✓	2,119 ms	333	958	820	**0**	15

Table 5: Deep private inference, width 16, weights *and* activations encrypted, real ABY3 protocol, BFV at $n = 4096$, $q = 2^{100}$. Linear in depth and bit-exact at every depth; the exactness is the evidence for the noise claim, since accumulated noise would make deep outputs diverge.

width	constraints	prove	verify	proof bytes
1	14	6.3 ms	1.0 ms	18,544
64	589	8.0 ms	1.3 ms	47,808
784	**7,159**	**32.3** ms	**4.8** ms	**111,712**

Table 6: Constraints grow linearly (≈ 9 per input); proof size does not—a 286× larger circuit yields a 6× larger proof. Width 784 is the MNIST input dimension.

and private inputs, and we prove it on a real Binius64 prover rather than on a placeholder. We give the results in Table 6.

We consider that two of our implementation findings generalize to anybody who builds arithmetic circuits over 64-bit words. The first is that comparison gates are **unsigned**, and therefore a signed comparison must flip the sign bits beforehand; without that step $\mathrm{ReLU}(-1)$ returns -1, which is to say that the circuit is perfectly sound about the wrong function. The second finding is sharper, and we state it as follows.

Finding. Division cannot be checked the obvious way. A circuit cannot divide, so the prover supplies quotient and remainder as witnesses; but checking $a = bq + r$ with $|r| < |b|$ is *unsound*, because circuit multiplication wraps mod 2^{64}. For $a = 1, b = 3$ a cheating prover sets $r = 0$ and $q = 3^{-1} \bmod 2^{64}$ and both checks pass. Requiring the 128-bit product's high word to be the sign extension of the low word closes it.

We remark that we drive that forgery through the real prover as a test, so that the soundness of the repair is demonstrated and not merely asserted.

6.6 Preprocessing: OT versus SHE

In this section we measure the preprocessing that Section 3.2 described, because it is where the comparison between homomorphic encryption and secret sharing is usually decided, and because it is usually decided implicitly. We give the two routes in Table 7.

A caveat about comparability that the literature routinely drops, and that we dropped ourselves. A prime field of approximately 60 bits does *not* represent every 64-bit value, and therefore Table 7 compares a triple over a 61-bit field against a triple over a 64-bit ring. At **equal width of** 64 **bits**, LowGear requires two limbs under the Chinese remainder theorem, which gives 384 B per triple against the 3,074 of the oblivious-transfer route. It means that the advantage in communication is **8.0**× and not the 16× that a naive reading suggests. Our own experimental record quoted 16.3×, and we correct it here. We remark that this is precisely the class of error that a single-harness study is positioned to catch, and that it is the reason why Section 8 exists.

route	domain	B/triple	triples/s	rounds
LowGear (SHE) [37]	$\mathbb{F}_p$, 60-bit prime	**192**	97,304	2
Semi2k (OT) [36]	$\mathbb{Z}_{2^{64}}$	3,074	2,681	3

Table 7: Two parties, semi-honest. **The two rows are over different domains**; see the caveat below for the apples-to-apples figure.

	MP-SPDZ [36]	Baru
Purpose	production-grade MPC across many protocols	one harness in which paradigms are measured against each other
Paradigms	secret sharing, garbled circuits, OT	the same, plus RLWE-based FHE, FHE↔MPC switching, and succinct proofs
Execution	compiler to bytecode → VM	CEK interpreter, protocol chosen per call at run time (Section 5.1)
FHE role	preprocessing (SHE-based triples)	preprocessing *and* a first-class evaluation paradigm
Breadth vs. depth	many protocols, one paradigm family	few protocols, four paradigm families

Table 8: Baru is not a smaller MP-SPDZ; it answers a different question. MP-SPDZ optimises *within* the secret-sharing paradigm and treats FHE as a preprocessing device. This study needs FHE, garbling and secret sharing to be interchangeable *evaluation* strategies for the same program so their costs can be subtracted.

The measured cost of the oblivious-transfer route also validates our analytic model, since we measured 3,074 bytes against 3,136 predicted, an error of 2 %.

7 Related work, and where this study sits

In this section we position our work with respect to the literature. We remark that organizing this literature by primitive would reproduce Section 3, and therefore we organize it instead by *what a system is for*, because in our view that is what determines whether the numbers of one system can be compared with those of another.

7.1 Frameworks that run many protocols

MP-SPDZ [36] is the closest system to this one in ambition and the natural point of comparison. It implements dozens of protocols — dishonest and honest majority, fields and rings, semi-honest and malicious, with the SPDZ [23], SPDZ2k [19], Semi2k and LowGear [37] families among them — behind a Python-like front end that compiles to a bytecode executed by a virtual machine. It is more complete than Baru on the MPC axis by a wide margin, it is the source of the triple-preprocessing figures in Table 7, and any practitioner choosing an MPC protocol should start there rather than here.

We consider that three differences matter for a survey, and we emphasize that none of them is a claim of superiority.

The first difference is the **scope of the paradigm**. MP-SPDZ is a framework for secret-sharing MPC, in which somewhat-homomorphic encryption appears as a means of generating triples. Our survey requires homomorphic encryption as an evaluation paradigm in its own right, because the

question of whether a given layer should be computed under homomorphic encryption or under secret sharing has no answer inside a system in which one of the two alternatives is only ever preprocessing.

The second difference is **compilation against interpretation**. A compiler is obliged to commit to a protocol in order to emit code, whereas an interpreter can leave the choice to a backend and consequently hold the program fixed across the paradigms, as we argued in Section 5.1.

The third difference is **maturity of the optimizer**, and here we were until recently describing our own artifact inaccurately. We had written that MP-SPDZ optimizes protocol performance while we optimize the comparability of measurements. The second half is true and the first half understates what we have: our compiler carries an instruction graph, an as-soon-as- possible levelling pass that collapses independent operations into one batched round, common subexpression elimination, reassociation of dependent reductions, per-protocol cost models, and an assignment pass that chooses a protocol per kind of operation. Those are the same categories of pass that make MP-SPDZ's compiler effective, and Table 9 measures what they buy against an expert baseline.

route	calls	**waits**	messages	bytes	ms (RTT 0)	ms (RTT 20)
hand-written gadgets	24	**33**†	61	33,466	53.2	493.2
hand-written, gc cmp	21	**12**	46	30,191	28.4	279.2
compiled, serial	15,304	18,362	18,367	3,822,372	26,344	245,972
+ ASAP levelling	15	43	47	881,324	195.1	757.3
+ CSE	15	43	47	33,504	61.0	620.7
+ reassociation	12	33	37	33,262	48.3	486.3
+ gc comparisons	9	**12**	22	29,987	32.4	269.9

Table 9: One network, $x(16) \to \text{dense}(8) \to \text{ReLU} \to \text{dense}(4) \to \max$ with secret weights, both routes over the same transport and both checked against the plaintext reference. The gadget route is written by an expert who called the batched primitives deliberately; the compiled route is scalar source with no batching hint. *Calls* counts backend primitives, *waits* counts the times a party blocked for a peer. The last rung is a protocol assignment rather than a compiler pass, so the hand-written route is given the same assignment in the second row — the two rows in bold are the comparison that means something. Bytes are one run and are stable; the millisecond columns carry the harness's run-to-run variance of roughly 13 %. †Waits carry a ± 1 of their own, and the marker is on this row because it is where we first saw it: over four repetitions at RTT 0 it read 32, 33, 32, 33, while every other row was identical in all four. A run that also swept RTT 20 then moved three rows by one count, so the jitter is not confined to this row — it is a property of the counter. A receive that finds its message already delivered does not block, so the quantity is how many times a party *had* to stop, and that depends on thread interleaving, which the network profile itself perturbs. Read every wait figure here as ± 1.

Finding. Automatic scheduling reaches *parity* with the expert, and the metric that shows it is neither of the two we reached for first. The compiled route makes half the primitive calls (12 against 24) and sends $0.61\times$ the messages, yet blocks the same number of times (33 against 32 to 33) and takes the same wall clock. Calls and messages do not predict cost here; waits do.

We should be precise about what "the same number of times" is doing in that sentence, because we were not at first, and we corrected ourselves twice. Waits are *almost* deterministic, not deterministic. The mechanism is visible in the instrumentation: a receive is counted as a wait only when it actually polls the transport, and one that finds its message already delivered does not, so the quantity is the

number of times a party *had* to stop, and that depends on how the three threads interleave.

Our first correction was to withdraw the phrase "exact parity", which we had written on the strength of two equal readings of 33. Four repetitions showed the expert's route reading 32 or 33 while every other row held. Our second was to withdraw the accompanying claim that only that one row was unstable: a later run that swept a second round-trip time moved three rows by one count each. The network profile perturbs the interleaving, so the jitter is a property of the counter and not of one route. Every wait figure we report should be read as ± 1.

What survives is the argument, because none of it rests on a single count. Parity is parity within the counter's resolution, and the effects we measure against it — a factor of 136 for levelling, 2.7 for the protocol assignment, 214 for the local-cost constant — are not one-count effects. We record the two corrections because the temptation each time was to quote the equality we had happened to observe.

The reason is that a primitive call is not a round. A `mul_batch` blocks once; a `lt_batch` performs an entire bit decomposition and blocks once per stage of the prefix adder. Counting calls therefore weighs a comparison and a multiplication equally, and a route that trades many multiplications for few comparisons looks better than it is. We had written this section twice on the strength of the call count, once claiming the compiler beat the expert and once attributing the residual to executor overhead; both readings were artifacts of the counter, and neither survived adding one that counts blocking.

We read the corrected table as good news of a narrower kind. Automatic levelling over a dependence graph, with no hint from the source, recovers what a careful author achieves by remembering to call the batched primitive — which is the premise the rest of this section needs. It also shows which pass does the work: levelling alone leaves the compiled route at 43 waits against the expert's 33, and it is reassociating the dependent reduction that closes the gap to 32. Common subexpression elimination contributes nothing to waits at all, and a great deal to bytes (881 against 33.5 kB), which is why the ladder is reported per pass rather than as one number.

The cost of a primitive, measured. Once blocking is the unit, the cost of each primitive can be measured rather than assumed, and Table 10 does so at two batch widths. The width columns agree, which is the property that makes levelling worth doing: a batched primitive blocks the same number of times whatever its width, so collapsing a layer into one call is free in rounds.

primitive	waits, width 1	waits, width 64	modelled rounds
`mul_batch`	2	2	1
`lt_batch`	9	9	7
`a2b_batch`	9	9	7
`reveal`	1–2	1–2	1

Table 10: Blocking cost per primitive at $\ell = 32$, against what our cost model charges

A comparison costs about four and a half multiplications, which is the ratio any scheduler trading one against the other has to know, and our model's structure already reflects it: it prices a comparison as an opening plus $\lceil \log_2 \ell \rceil + 1$ multiplication rounds, against one for a multiplication. The constants have not been fitted — the model charges 1 where the implementation blocks twice, and 7 where it blocks nine times — so the ratio is the right order (7× modelled against 4.5× measured) while the absolute figures are not yet calibrated. Now that blocking is counted, they can be.

We also record a place where the model and the executor are defined differently, together with

what that costs, because the second part is easy to assume and was not what we assumed. The model charges a level the *maximum* over the primitives it contains, on the reasoning that a level is one logical round; the executor issues one batched call per *kind* per level and each blocks, so a level holding a multiplication and a comparison pays their *sum*. We expected this to be the largest item outstanding. Counting the mixed levels in the network of Table 9 shows there are *none*: the schedule has 8 interactive levels and issues 8 batched calls. The reason is structural rather than lucky — a ReLU written as $z \cdot (z > 0)$ puts its comparison and its multiplication at consecutive levels by construction, and so does a mux-style max — so the two definitions differ and this class of program does not pay for the difference. We keep the counter (`ExecStats::mixed_levels`) so that a schedule which does pay is visible rather than inferred, and we do not claim the optimisation.

Which primitive to attack, and the assignment that follows. Attributing the 32 waits to levels says where the budget goes: three of the eight interactive levels hold a comparison, and at nine blocks each they account for 72 % of the total, against two blocks for a multiplication level. The comparison is therefore the primitive worth changing, and Baru has three of them. Table 11 measures what each costs.

comparison backend	waits	messages	kilobytes
`a2b` — batched bit decomposition	10	16	127.1
`falcon` — Π_{DReLU} over $\mathbb{Z}_p$	11	44	**49.1**
`gc` — three-party garbled circuits	**3**	11	109.9

Table 11: 64 comparisons at $\ell = 32$, online phase, preprocessing filled in advance. Bytes are the whole run including preprocessing.

The spread is 3.7× in blocking, which is what makes an assignment worth deriving at all: if the three agreed there would be nothing to choose between them. Two things follow. The first is that `gc` ***strictly dominates*** `a2b` here, in blocking and in bytes both, and `a2b` was the default the compiled route used — so the assignment pass had a free improvement available that no correctness test and no byte count would ever have surfaced. The second is that `falcon` is the byte-cheapest by 2.2× while being the block-dearest, which is the same trade our crossover measurements report at the network level (§9.3) arriving here at the level of a single primitive. An assignment cannot be read off one column.

Finding. Assigning the comparison levels to the constant-round backend cuts the compiled route from 33 blocks to 11 to 12, a reduction of 2.75 to 3.0 times and the largest single lever after levelling itself. It is also *not a compiler result*: giving the hand-written route the same assignment takes it from 32 to 33 to 11 to 12 as well, so measuring the assigned compiler against the unassigned expert would report roughly a 0.36× win of which the compiler contributed nothing. Compared like with like the two are at parity, and they are at parity in the strong sense that their observed *ranges* coincide over eight runs rather than that two particular readings happened to agree.

Keying the choice on the deployment, and the third term that decides it. An assignment that is right for one link is not an assignment. Baru's precompile registry previously held one champion per gadget, set by a maintainer on the strength of a benchmark; it now also answers *which entry for this link*, ranking the measured candidates under the latency model. We built that expecting Table 11 to demonstrate it: `falcon` is 2.2× cheaper on the wire and 3.7× dearer in

blocking, so the champion ought to flip between a thin pipe and a slow one. We then measured all three backends across the grid, and report the result because it is not the one we expected.

backend	network free	RTT 0, 10 Mbit s^{-1}	RTT 40, 100 Mbit s^{-1}
`a2b`	33 ms	206 ms	334 ms
`falcon`	79 ms	130 ms	1,056 ms
`gc`	**13** ms	**105** ms	**201** ms

Table 12: The same 64 comparisons at $\ell = 32$, wall clock, across three links. The first column is `NetworkProfile::INSTANT` — zero latency, unlimited bandwidth — so it isolates local cryptographic work.

Finding. Garbled circuits are fastest at every point of the grid, and the reason is a term our cost model does not contain. With the network free, Falcon's comparison costs 6× what the garbled one does at this batch size — that is local arithmetic over $\mathbb{Z}_p$ together with its preprocessing — and a model of the form waits × RTT + bytes/bandwidth charges every candidate zero for it. The true ratio is larger still, and we arrive at it below: measured on a whole diagnosis rather than on 64 comparisons, the factor is 16.4 (Table 13).

The omission is not cosmetic, and it failed in two distinct ways. On a fast link the model predicts 0 s for all three candidates and cannot rank them at all, so the choice fell through to a tie-break which returned the *dominated* entry — a wrong answer rather than an arbitrary one. On a thin pipe it mispicked outright, preferring Falcon at 130 ms to garbled circuits at 105 ms: the 78 kB Falcon saves is worth 62 ms at 10 Mbit s^{-1}, and it spends 66 ms more in local work to save it. A measured local term restores agreement with the wall clock to 1.00× at all three points.

The microbenchmark understates the spread, and we nearly built on it. Table 12 measures 64 comparisons in a fresh session, which is the natural thing to measure and the wrong denominator. Measuring the same comparison work inside a whole diagnosis — 7,137 comparisons, the workload the assignment exists for — gives the nonlinearity compute directly, and it does not agree.

backend	in a diagnosis	implied marginal, 64 wide	microbenchmark, 64 wide
`gc`	137 ms	1.2 ms	13 ms
`a2b`	1,063 ms	9.5 ms	33 ms
`falcon`	2,248 ms	20.2 ms	79 ms
spread	1 : 7.8 : 16.4	1 : 7.8 : 16.4	1 : 2.5 : 6.1

Table 13: Local comparison cost measured two ways. The middle column divides the first by the comparison count and rescales; the right column measures 64 comparisons on their own.

A 64-wide call costs roughly ten times its own marginal cost, because most of it is session setup and share distribution rather than comparing. The absolute inflation would not matter — a ranking does not read absolute numbers — but it is not uniform across the candidates, and the ratio it distorts is exactly what a ranking does read. It makes garbled circuits look 2.5× cheaper than bit decomposition where the marginal figure is 7.8×.

This is the same slope-against-intercept error we corrected in the analytic model's local-work constant, in a different instrument, and we record it as a pattern rather than as two incidents: ***a***

cost measured at one scale and applied at another is an intercept masquerading as a slope, and the symptom is that the fixed part is common while the ratio is not.

We then made the correction, and it was wrong. Substituting the marginal figures for `local_secs` while leaving the byte term alone flipped the selection to Falcon on a thin pipe, where measurement says garbled circuits win — caught by the validation test rather than by inspection. The error was making one term marginal and not the other, and the byte term does not decompose the same way: Falcon's traffic includes *its own* preprocessing, which is a protocol-specific cost and not a fixed one common to the candidates. There is no common term to strip. We reverted to validated totals and kept the marginal measurement as a statement about direction: every correction to the ratio favours the champion already selected, so the assignment is more right than the registry's numbers imply and never less.

We keep the link-keyed machinery and withdraw the example. The machinery is the right shape, the byte-against-blocking trade it expresses is one we measure at the level of whole routes (§9.3), and the assignment it selects is worth 2.75 to 3.0 times in blocking, the range being the counter's own ±1. What we no longer claim is that the comparison gadget is where the champion flips: it is not, and the test that asserted a flip now asserts the measured fact, so a change that produces one will fail it and be looked at.

The same term in the analytic model, and what fitting it changed. Our analytic cost model does not have the omission just described — it carries a local-work term and reports a *compute* bottleneck when that term dominates. Its problem was narrower and worse: the constant feeding it had carried the comment *"an estimate; calibrated in P1"* since it was written, and that calibration never happened. Our own calibration command validates predicted rounds and predicted online bytes against a measured run and reports zero error on both; it never touched compute. So we fitted it, and we describe the method because the number is machine-specific and the method is not: wall clock on a free network is per-lane work *plus* a fixed cost — thread spawn, share setup — so dividing one measurement by its width charges the fixed part to every lane. We measure four batch widths from 1,024 to 65,536 and fit the ***slope***, reporting the intercept separately; Table 14 gives the result together with the two quantities that bracket it.

quantity	measured
`mul_batch`, real transport, network free	**1,072** ns per lane
local-only `add`, no communication at all	145 ns per lane
the estimate it replaced	5 ns per lane

Table 14: Fitting the per-lane local cost of an ABY3 multiplication

Finding. The assumed constant was 214 times low for a multiplication lane, and 29 times low even for the arithmetic floor: an addition, which communicates nothing whatsoever, already costs 145 ns in share bookkeeping and a ring addition. A multiplication is 7.4× an addition, the remainder being serialization and the channel round.

Neither reading rescues the constant, so we replaced it and marked it fitted, leaving every other protocol's at its literature estimate and marked as such. The correction is not cosmetic. On a three-instruction program compute is 3 µs against 878 µs of latency and nothing moves; on a *batched* program — 513 interactive operations over two dependence levels, which is the shape of a dense layer — the predicted total moves from 169 µs to 716 µs, a factor of 4.2, and the reported bottleneck flips from latency to compute. That is the regime every neural-network claim in this paper lives in.

We are obliged to add that the resulting table is *not* a ranking, and we have made the artifact say so. ABY3 is now the only protocol whose compute term is measured; the others carry estimates that are presumably low by a comparable factor, so reading a cross-protocol ordering off a compute-bound table would be the same error of comparing incommensurable quantities that this section has already recorded three times. The comparison output now emits a note whenever any row is compute-bound, naming which constant is fitted and declining the ordering. We also record that our three-party *simulator* is unusable for timing — it has no batched multiplication and spawns three operating-system threads per multiplication, measuring 58 µs per lane against the real backend's 1.07 µs — because it is the instrument one would naturally reach for, and the calibration test now says so at the point of temptation.

We labour the control because this benchmark's own history is three readings that compared quantities which were not comparable, and the test that fixes the fourth now asserts that the mismatched comparison is the flattering one — so the reading that would overstate the compiler is pinned in place as a thing to avoid rather than described in a comment. What the result does establish is the premise the next section needs: the comparison backends differ enough to be worth choosing between, the choice is invisible to correctness, and the compiler is a good enough consumer of such a choice that it will not squander it.

We remark that this is a narrower claim than MP-SPDZ's, and deliberately so. MP-SPDZ implements tens of protocols against our handful, its compiler vectorizes where ours does not, and its engineering is years ahead; a practitioner choosing an MPC protocol should still start there. What Table 9 supports is only that automatic scheduling over a dependence graph recovers, and slightly exceeds, what a careful author achieves by remembering to call the batched primitive — which is the premise the rest of this section needs, because a per-operation protocol assignment is worth deriving only if the code that consumes the assignment is not itself leaving rounds on the table.

Frameworks based on circuit compilation. ABY [24] introduced mixed-protocol two-party computation with explicit conversions between arithmetic, Boolean and Yao sharings. HyCC [10] automated the decision of *partitioning* a program, compiling ANSI C into a mixed-protocol program. CirC [48] unified circuit compilation across MPC and proof back ends, and Silph [14] treats the assignment of protocols as an optimization problem. Our own cost model, which we describe in Section 9, is a small instance of this line, and we report there that the hybrid split for a neural layer can be *derived* from measured per-operation costs rather than annotated by hand. We remark that the distinction from those systems is again interpretation: they choose an assignment and emit a circuit, whereas we choose per call and keep the program.

7.2 Three-party honest-majority protocols

ABY3. The work of Mohassel and Rindal [46] is the protocol layer that our artifact reimplements, namely replicated sharing over rings, the triple-free multiplication of Theorem 3.2, conversions among arithmetic, Boolean and Yao representations, and fixed-point primitives for machine learning. For our purposes its central insight is that with three parties and one corruption the *online* cost of a multiplication falls to one ring element per party and requires no preprocessing at all, and it means that honest-majority secret sharing becomes competitive with homomorphic encryption on linear algebra.

Falcon. On that base, Wagh et al. [55] construct a set of protocols for the nonlinearities, and we reproduce them in detail: private comparison over a small prime field $\mathbb{Z}_p$, the `wrap3` computation

of the wrap of three summands, Π_{DReLU} and Π_{ReLU}, the share selection Π_{SS}, max-pooling with a one-hot indicator, and exact truncation. Section 9.3 is largely a reproduction study of these protocols, and we consider it the part of this survey in which reproduction produced the most, because matching a published protocol at its own ring width and on its own network obliged us to separate the offline cost from the online one, and that separation changed our conclusion twice.

We flag two details for anybody who reproduces this work. The first is that the ReLU of Falcon carries a one-hot indicator because Falcon also *trains*, and consequently charging an inference-only pipeline for that indicator is not a fair comparison. The second is that its exact truncation states a bound on the mask which is necessary for correctness and insufficient for privacy, in the sense of Definition 3.2; we describe the consequence in Section 12.

Other work in the honest-majority setting. SecureNN [54] preceded Falcon with three-party protocols that avoid garbled circuits for comparisons. ASTRA [12] and the works of Chida et al. [16] and of Eerikson et al. [27] supply the machinery for malicious security that we described in Section 3.2, and SING [5] continues the thread of nonlinearity protocols. We implement the MAC construction of Chida and Eerikson and we measure its cost against the semi-honest baseline rather than assuming it.

7.3 FHE-based and hybrid inference

The space of hybrid designs is where the largest differences that we measure in this survey are found. Gazelle [35] established the pattern, namely linear layers under lattice-based homomorphic encryption with packing, nonlinearities under garbled circuits, and conversions between the two. Delphi [45] moved work into an offline phase and traded accuracy for speed by means of polynomial activations, and MiniONN [43] had taken an earlier mixed approach. CrypTFlow2 [49] contributed protocols for comparison and truncation with substantially better constants, and it is the reference point toward which our own findings repeatedly point. Cheetah [32] contributed the coefficient-packed convolution of Section 3.4, which requires no rotations, and it is the technique whose exactness we verify at 5×5 in Section 10. We remark that PEGASUS [34] and CHIMERA [7] address switching *between homomorphic schemes* rather than between homomorphic encryption and secret sharing, which is a different problem from ours.

Our contribution relative to this line is not a faster hybrid. It is that the hybrid route, the pure homomorphic route, the pure secret-sharing route and the garbled route are all present in a single artifact over a single transport. It means that the question of what the hybrid actually buys, and of where the crossover lies, can be answered by measurement rather than by comparing numbers taken from four papers with four experimental setups.

7.4 Homomorphic encryption schemes and libraries

BFV [29] and BGV [8] are the two schemes that we implement, in the encodings of Definition 3.9. CKKS [15] targets approximate arithmetic over the reals and we do not use it here, because our results depend on bit-exactness against a plaintext reference. TFHE [17, 53] targets fast Boolean bootstrapping and is present as a backend for comparison. SEAL [51] and NTL [52] are the reference implementations against which any work of this kind is measured, and we make no claim of competing with them on constants, as we state in Section 12. Our selection of parameters follows the standard for homomorphic encryption [1] and the line of work on lattice estimators [2, 26, 38].

We remark that our one negative result in this area is a scheme that we *removed*. It was an NTRU variant in the style of LTV and YASHE [44, 6], it was fast, and at the parameters that our

construction requires it was not secure; we describe the episode in Section 11.

7.5 What is distinctive here

No individual protocol in our artifact is novel, and we claim none. We believe, however, that three elements are unusual in combination.

1. **Four paradigms behind one abstraction, in one artifact and over one transport**, with a single program text reaching all of them, as we argued in Section 5.1. The existing frameworks are deep in one family of paradigms and they treat the others as preprocessing or omit them entirely.

2. **Comparability treated as the object of study.** The uniform accounting, the bit-exactness gate and the validated cost model for the network are themselves the contribution, and we remark that several of our findings are corrections to comparisons rather than new speedups; we present them in Sections 8 and 11.

3. **Negative and withdrawn results kept in the record**, with the component deleted in the cases where it lost. We report three optimizations that were measured and rejected, four successive wrong answers to a single question about a crossover, and two gaps in the security of our own code, one of which we created while implementing a published protocol faithfully enough to inherit an under-specified precondition.

8 Methodology

In this section we present our methodology. It is the part of this work that we would most like other artifacts to adopt, and consequently we state each element as a rule together with the error that the rule exists to prevent.

8.1 Identical transport, uniform accounting

Every protocol, whether arithmetic secret sharing, garbled circuits, the Falcon reproduction or the hybrid, runs over the same transport stack, which is wrapped in counting and latency decorators that observe the traffic without touching the code of the protocol. It means that both legs of any comparison pay a real serialization, a real passing of messages and a real counting of bytes, and consequently there is no asymmetry in which one side is simulated and the other is not.

We remark that this matters more than it appears to. Our in-process simulator *is not the protocol*, since its multiplication of shares reconstructs and reshares locally, which is free and insecure, and it understated the pure secret-sharing baseline by approximately 2.7×, namely 6.7 ms simulated against 18 ms real. Every number that we report for secret sharing in this survey is the real wire protocol, with three party threads that exchange serialized messages.

8.2 A validated network model

Our cost model charges rounds × RTT + bytes/bandwidth, which allows a benchmark to move the parties between continents without deploying anything. We **validated it against a real loopback TCP socket**, and we measured a worst-case error of 13.6 % and an error below 1 % through the middle of both sweeps, with the constants of the link measured rather than assumed. It means that

the cost really is linear in the rounds and linear in the bytes, with no missing cross-term, and this is what licenses us to compute a wall-clock figure from measured counts.

Constructing this validation revealed two defects in our harness that had been silently distorting results. The per-message latency timers could *reorder* back-to-back messages, and the bandwidth model did not serialize bulk transfers cumulatively, which *undercharged precisely the byte-heavy phases* that the crossover experiments are about. Both defects now have regression tests. We mention them because a cost model that has never been checked against a socket should be assumed to contain defects of this kind.

We state the limitation openly: the constants are those of loopback, with a round-trip time of 23 µs, and not those of a wide-area network, and therefore a wide-area deployment requires its own two constants.

8.3 Bit-exactness as a gate on quoting cost

Finding. A cost comparison between routes computing different things is meaningless.

Consequently we check every backend against a common plaintext reference before any number is reported, and our benchmarks refuse to present a clean table when a cell disagrees. The encrypted convolutional network reproduces the plaintext prediction on 600 of 600 embedded medical images, and we ran it twice, before and after two optimizations, with both passes agreeing on all 600 images. It means that those optimizations are *exact* and not merely close.

8.4 Machine-checked security attestations

Every engine attests the security level that it actually implements, in a single auditable table, and asking a session for more than an engine attests is an *error rather than a silent downgrade*. We remark that exactly one of our engines attests malicious security. The editing rule is that an attestation records what the code *does*, never what the roadmap intends: raising one is the last step of an upgrade, after the cheating-detection tests pass.

We consider this the most transferable idea in the artifact. Security level is the dimension most often lost when numbers move between papers, and making it a machine-checked property of running code—rather than prose in a limitations section—is cheap to implement and hard to get wrong by accident.

8.5 Recording withdrawn results—and deleting the component

We keep negative, superseded and withdrawn results in the experimental record beside the successful ones, together with the reasoning that produced them, and we summarize what this practice caught in Section 11.

We remark that the half of the rule which is easy to omit is the other one: **when a measurement condemns a component, the code is removed and the number is retained**. An artifact that accumulates every paradigm it has ever measured becomes unmaintainable, and worse, it becomes misleading, because a reader assumes that everything present is recommended. On this basis we removed the following components. Single-server private information retrieval, which loses to downloading the whole database by approximately 13× and admits *no* crossover in the database size N. The SEAL and CKKS backends, whose approximate comparison costs seconds per operation and carries a real error, against milliseconds and bit-exactness under secret sharing, and which carried 3.4 MB of vendored C++. An NTL polynomial backend, which our hand-written transform

width	dealer (lower bound)	Semi2k (OT)	**LowGear (SHE)**	hybrid/BFV
64	600.6 ms	694.5 ms	**606.2** ms	692.1 ms
1024	603.0 ms	942.8 ms	**623.4** ms	692.1 ms
4096	610.9 ms	1.74 s	**678.4** ms	733.7 ms

Table 15: One private layer with both operands secret plus a ReLU; WAN, two parties, preprocessing charged per triple *consumed*.

beat by 1.8× at every size. And an unsound placeholder for the proof system, which we describe in Section 11.

Two consequences follow. The first is that **the figures given here for a removed component are not reproducible from the current artifact**, and our experimental record says so at the point of measurement. The second is that where a removed measurement still feeds a live claim, it survives as the smallest object that can carry it. In other words, the bootstrap counts attributed to a levelled competitor in Table 5 come from the level arithmetic of the CKKS sign approximation, which is now a module of forty lines with no dependencies, and whose figures are pinned by a test precisely because the apparatus that produced them has been deleted.

9 Cross-paradigm findings

In this section we present the findings that motivated the single-harness approach, in the sense that the literature of no single paradigm could have produced them. In Section 9.1 we show that the crossover between homomorphic encryption and secret sharing belongs to the preprocessing of the baseline. In Section 9.3 we locate the crossover between the two routes for the nonlinearities, and we report the four answers that we gave before the present one.

9.1 The FHE-versus-MPC crossover belongs to the baseline's preprocessing

The standard claim is that homomorphic encryption overtakes secret sharing beyond some width of the layer, because the preprocessing of secret sharing grows linearly whereas a packed ciphertext is flat. Our cost model predicts this behaviour and our measurements confirm it, but we remark that they confirm it *against a baseline preprocessed by oblivious transfer*. We give the figures in Table 15.

Against secret sharing preprocessed by **somewhat-homomorphic encryption**, the crossover disappears from this range entirely, and LowGear tracks the lower bound given by a trusted dealer to within 5 ms at width 64.

Finding. The honest statement is not "the hybrid beats dishonest-majority MPC past width 64" but "the hybrid beats *OT-preprocessed* dishonest-majority MPC past width 64, and loses to SHE-preprocessed MPC at every width measured." The mechanism survives—preprocessing volume decides it—but the boundary is a property of the baseline, not of either cryptosystem.

A second boundary moves with the **threat model**. Under an honest majority, that is, with the ABY3 multiplication of Theorem 3.2 from correlated randomness and no preprocessing at all, the layer remains on secret sharing at *every* width. Under a dishonest majority with preprocessing from oblivious transfer, homomorphic encryption takes it beyond approximately 256 multiplications. We consider that this explains a structural fact about the literature which is otherwise puzzling, namely that *every hybrid inference system combining homomorphic encryption and secret sharing is two-party*. Gazelle, Delphi and Cheetah are situated precisely in the regime in which the model says that the hybrid pays.

RTT	protocol	1000	100	40	30	20	10 Mbit s^{-1}
0 ms	gc	**558**	**2983**	**6669**	8,569	12,281	23,840
	falcon (cold)	2,588	4,529	7,259	8,646	**11483**	**20235**
	falcon (warm)	995	2,760	**5069**	**6359**	**8915**	**16552**
20 ms	gc	**2414**	**4671**	**8154**	**10067**	**13940**	**25421**
	falcon (cold)	9,142	10,818	13,488	14,964	17,732	26,428
	falcon (warm)	4,998	6,431	8,807	**10087**	**12681**	**19706**

Table 16: Milliseconds per encrypted MedMNIST diagnosis, median of three repeats; winner in bold. *Cold* charges Falcon for preprocessing on the critical path; *warm* generates it per MPC segment beforehand and reports the online cost. Garbled circuits have no query-independent phase to hoist, so their row serves both. Cold crossover between 20 and 30 Mbit s^{-1} at RTT 0 and below 10 Mbit s^{-1} at 20 ms; warm, between 100 and 1,000 Mbit s^{-1} and between 30 and 40 Mbit s^{-1}. Higher round-trip time moves each boundary *down*; hoisting preprocessing moves it sharply *up*. Cells within ≈13 % are unresolved (Section 12): at RTT 0 the 30 Mbit s^{-1} cold cell and the 100 Mbit s^{-1} warm cell are ties, so each boundary is a band.

9.2 Protocol assignment can be derived, not annotated

Given the measured costs of each backend, a search over the assignments of protocols to kinds of operation returns `mul` → fhe and `cmp-lt` → binary for an encrypted layer of width 256 containing a comparison. In other words, the search *derives* the architecture of Gazelle, Delphi and Cheetah instead of receiving it as an annotation. We remark that the prior work on the assignment of protocols, from HyCC [10] through Silph [14] to SING [5], is **restricted to secret sharing**, and that extending the cost model so that it spans secret sharing *and* homomorphic encryption is what makes the hybrid architecture an output of the search rather than an input to it.

Two structural non-linearities make this objective non-separable, which is worth noting for anyone tempted to formulate it as an integer linear program: a dependence level costs the *maximum* over its instructions, not their sum; and conversions *batch per level*, so a crossing's cost depends on whether another crossing already happens there.

9.3 The nonlinearity crossover, and three wrong answers on the way to it

We implement three interchangeable protocols for comparison behind a single interface, namely batched bit decomposition, garbled circuits, and the Π_{DReLU} of Falcon [55] based on `wrap3`, and we verify that the three compute the same function bit-exactly. In Table 16 we measure them with *both* protocols charged for their preprocessing and with neither of them paying for serialisation padding.

The mechanism appears in the same run. On the party-to-party transport Falcon sends 3.4 MB against the 10.1 MB of garbled circuits, which is 2.94× cheaper, and it does so in several hundred messages against 61, which is around 7× dearer. It means that the two terms of the cost model trade against each other, and that each boundary is located where the trade balances.

We must qualify the byte figure, because we quoted it for some time without the qualification and it changes how decisive that axis is. The party-to-party transport is not all the traffic: the hybrid also moves FHE ciphertexts and scheme-switch payloads on the ***evaluator link***, which does not pass through the instrumented party transport and so appeared in no counter, while the latency column paid its bandwidth cost throughout. Fitting a diagnosis's time against the reciprocal of bandwidth exposed the gap — the fit is clean and linear, with a slope near 29 MB where the traffic table said 10.1 MB — and the missing term is 17.0 MB per diagnosis, identical for both routes

because it is the same pipeline either way. Total traffic is therefore 27.0 MB for garbled circuits against 20.4 MB for Falcon.

Finding. Falcon's byte advantage is 1.33× on what the deployment actually moves, not the 2.94× the party transport alone reports — a common additive term of 17 MB dilutes it by 2.2×. Because the term is common, the crossover *location* is unchanged: adding the same quantity to both curves cannot move where they cross. What changes is the magnitude of the advantage, and hence how strongly bandwidth ought to be expected to favour one route.

We remark that pooling the preprocessing also *reduces* the messages of Falcon, because a single draw per segment batches better than many inline ones, and since its bytes are unchanged, warming the server improves both terms at once. Falcon's message count is no longer a fixed number, and we give it as a range above for that reason: closing the private-comparison leak of §3.5 replaced an opening of the mask with an opening of a masked product, and the rejection sampling that needs adds a data-dependent number of retries. Two consecutive runs of the identical configuration measured 474 and 482.

We reported three other answers before this one, and we consider the sequence to be the more useful result. Each of them was reproducible, each computed the correct diagnosis, and each agreed with our cost model; the first even matched a prediction of the break-even point at 17.1 against the 15 to 20 that we measured.

1. 15–20 **Mbit s^{-1}.** Falcon's Π_{WA} preprocessing was dealt from a shared seed, free precisely because it was not secret—so this compared Falcon's *online* cost with garbled circuits' *total*.

2. **No crossover at any bandwidth.** With a real offline phase Falcon measured 32.5 MB against 10.1 MB and appeared dearer in both terms. **82% of those bytes were padding**: Baru serialised ring shares as `u128` whatever their width.

3. **A tie.** The grid stopped at 10 Mbit s^{-1}, above the boundary.

We remark that the first answer was wrong in *both* directions at once, since dealt preprocessing flattered Falcon whereas the padding penalised it. Correcting both errors moves the boundary *upward*, and consequently Falcon wins over a wider region than we originally claimed, for entirely different reasons. In other words, the two errors had partly cancelled, and this is why the figure appeared sound.

Neither the bit-exactness gate nor the agreement with the cost model detects any of the three errors. Both of those mechanisms establish that a computation is correct and that its cost is explicable, and neither of them asks whether two measurements are *commensurable*. What caught these errors was decomposing each total into the terms that the model charges, namely the phases, the messages, the bytes and the padded share of the bytes, and then verifying that both protocols were charged for the same things.

We now apply one further rule generally: a spot check is not evidence about an encoding. Narrowing the ring wire to a fixed width of 21 bits was exact for the dataset whose activations fit in 2^{20}, and it was wrong for *every* image of the dataset whose activations reach 2^{22}, because the safe width is a property of the model and not a constant.

What we can still say. The architectural claim survives and is worth separating from the performance one:

Finding. Three implementations of one comparison, bit-exact against each other behind a single interface, make the protocol backing every ReLU, max-pool and argmax a one-line choice rather than a rewrite—which is what made this error detectable rather than shippable.

The per-deployment claim, withdrawn and then restored by measurement. We briefly concluded the opposite of what we had claimed: that there was *nothing* to choose on this workload, because garbled circuits dominated everywhere we had measured. That conclusion rested on a limitation rather than on a result. `lt_falcon_batch` ran its preprocessing inline, so every figure in Table 16 was a *total*, and Falcon is a protocol designed to put work in a query-independent phase—charging it inline is charging it for the thing its design exists to avoid.

Building the split was the open work, and it is now built: preprocessing is generated per MPC segment from a plan a cold run *measures* (sizes cannot be derived from the layer shapes without duplicating the batching decisions inside max-pooling, the fused ReLU and argmax), and a pool that fails to serve a request is a hard error rather than a silent fallback, so an "online" latency cannot quietly contain preprocessing. The warm row of Table 16 is the result, and it restores the per-deployment claim in a stronger form: a *cold* server should prefer garbled circuits above $\approx$25 Mbit s^{-1}, a *warm* one should prefer Falcon almost everywhere at low round-trip time, and the gap between those two answers is a factor of four in bandwidth.

Falcon's own claim—that its *online* phase is cheap—is now testable too, and it holds. At $\ell = 32$ on Falcon's own LeNet workload our online phase is 0.25 MB per party against a 2.87 MB total: *below* the 0.74 MB they report, while our total is several times above it. Which comparison is the fair one depends on whether their figure includes preprocessing, which we cannot determine from the paper, so we publish both and call neither the answer.

Two asymmetries keep this honest. Garbled circuits gain nothing from warming—they have no query-independent phase here—so the warm boundary must be read *alongside* the cold one and never instead of it; presenting only the warm row would reproduce exactly the error that produced our first withdrawn crossover. And our byte counters are whole-run totals, so in a warm sweep the latency column is an online cost while the traffic figures remain totals; the two are not on the same footing and our tool says so in its own output.

The scope of the withdrawal. One bit decomposition per element is an artefact of *our* construction rather than a lower bound. Writing $w_{01} = \mathrm{wrap2}(x_0, x_1)$ and $y = (x_0 + x_1) \bmod L$, one gets $\delta = w_{01} \oplus [\, y + x_2 \geq L \,]$ (verified exhaustively at $\ell = 6$). Party 0 holds both x_0 and x_1, so it computes w_{01} and y locally and for free, leaving a single comparison between a value it knows and a value parties 1 and 2 know—a millionaires' problem, not another Π_{WA}, and one that the same no-OT three-party garbling used for the `gc` route settles in one round and $\approx\ell$ AND gates. We have not implemented it and claim no cost for it. It is enough to fix the scope: what we withdraw is *our* crossover, and the correct reading of Table 16 is "Baru's Falcon loses to Baru's garbled circuits once preprocessing is charged"—not that Falcon's DReLU fails to repay its preprocessing, which these measurements do not show and the construction above suggests is false.

10 Case study: private medical image diagnosis

In this section we give a case study. MedMNIST v2 [57] distributes twelve biomedical datasets at 28×28, which is exactly the shape of MNIST [42], and consequently a pipeline for private inference runs on them without modification and our artifact ceases to be a statement about handwritten digits. We give the results in Table 17.

	PneumoniaMNIST	OrganAMNIST	DermaMNIST
task	chest X-ray, 2 classes	abdominal CT, 11 organs	dermatoscopy, 7 lesions
input	$1 \times 28 \times 28$	$1 \times 28 \times 28$	$\mathbf{3 \times 28 \times 28}$ (RGB)
integer accuracy, full test set	84.78 % (624)	**82.93** % (17,778)	72.12 % (2,005)
95 % Wilson CI [56]	$[81.7, 87.4]$	$[82.4, 83.5]$	$[70.1, 74.0]$
balanced accuracy / AUC	80.64 % / **0.9314**	**82.21** % / **0.9605**	30.14 % / **0.8519**
sensitivity / specificity	97.18 % / **64.10** %	—	—
published ResNet-18 (28×28)	0.854 / 0.944	0.935 / 0.997	0.735 / 0.917
majority-class baseline	62.50 %	18.48 %	**66.88** %
encrypted = plaintext	**300/300**	**300/300**	**300/300**
FHE bootstraps	0	0	0
online latency / diagnosis	**268** ms	291 ms	271 ms

Table 17: Private diagnosis on real biomedical images, image *and* weights encrypted, only the class index revealed. DermaMNIST is read on balanced accuracy and AUC: its largest class is 66.88 % of the test set, which published ResNet-18 beats by under seven points. Its RGB input costs nothing extra—an RGB 28×28 convolution needs degree 3,977 against $n = 4096$ and so fits one ciphertext at one multiplication per output channel, which is why its latency matches the grayscale rows.

We consider that three points deserve the attention of a privacy venue.

Privacy costs nothing in accuracy, and we show this rather than asserting it. The integer model matches its floating-point parent to within 0.2 points, and the encrypted pipeline reproduces the plaintext diagnosis bit-exactly on every one of the 600 images.

Accuracy alone misreads a medical classifier. The test set of PneumoniaMNIST is approximately 60% positive, and our model earns its 84.78 % largely by *over-calling the disease*, since it attains 97.2 % of sensitivity against 64.1 % of specificity, and its balanced accuracy is 80.64 %. None of this is visible while only the accuracy is reported, and we remark that none of it is a cryptographic result; nevertheless a paper that claims private diagnosis is obliged to report it.

The ceiling on accuracy is the packing and not the cryptography. We initially attributed the size of the model to the noise budget of the homomorphic scheme, and that was wrong. Three limits bind, and the noise is the third of them. Spatially, a valid convolution of size 3×3 followed by pooling takes $28 \to 13 \to 5 \to 1$, which admits at most three convolutions. By *packing*, the constraint $2 \cdot \mathrm{in}_c \cdot H \cdot W \lesssim n$ caps the first convolution at 12 channels, before the input of the second one exceeds $n = 4096$. Depth, in contrast, really is free, for the reason given in Section 6.4.

We have since lifted that limit on packing without enlarging the ring. A convolution is linear in its input channels, and therefore a layer that is too wide for a single ciphertext can be evaluated as equal groups of channels whose products are *added*, which is free homomorphically and requires neither an evaluation key nor any depth. We remark that doubling the ring to $n = 8192$ would also have worked, and that it would have charged every operation of the pipeline, whereas splitting the channels charges only the layers that overrun.

It buys accuracy on one of the two datasets, and the difference is instructive. Retraining both at $\mathrm{conv}(16, 32)$—whose second convolution takes 16 channels at 13×13, degree 5,266 against $n = 4096$, hence two groups of 12—moves organa from 81.64 % $[81.1, 82.2]$ to **82.93** % $[82.4, 83.5]$, disjoint intervals on 17,778 test images. Pneumonia does not move: 84.78 % to 83.97 %

with overlapping intervals, its validation accuracy rising while its test accuracy falls. Both remain bit-exact with zero bootstraps.

So "the packing was the accuracy ceiling" holds *conditionally*, and the condition is data: organa's 34,561 training images can use the extra capacity, pneumonia's 4,708 overfit it. Either experiment alone would have been misleading.

11 What the methodology caught in our own work

In this section we record what the methodology of Section 8 caught in our own work. We consider that the credibility of a study rests partly on what its authors were willing to withdraw, and consequently each item below is recorded in the artifact rather than quietly fixed.

A backend for proofs that proved nothing. Until 2026-07-31 the `verify` routine of the shipped backend never checked that a witness satisfied any constraint, and it means that anybody holding the public circuit could forge an accepting proof. We retain the routine, unused, so that a soundness test can keep demonstrating precisely what it failed to do. **Every figure for the proof system that predates that date is withdrawn.**

A break of privacy in bit decomposition. Our previous `a2b` masked its input with a value derived from a *public* lockstep counter, and therefore the input was recoverable from the transcript by any observer. Everything built upon it, that is, ReLU, max-pooling and argmax, leaked its activations. We remark that the advertised property of requiring no extra communication per bit was in fact a *symptom*, since the carry chain was free precisely because every operand was already public. Repairing the protocol cost 4.4× in time and approximately 130× in bandwidth, and we later recovered both by means of the triple-free multiplication of Theorem 3.2 and of batching.

An insecure homomorphic scheme, removed rather than caveated. Our fast engine was a port of a scale-invariant NTRU construction [44, 6]. An analysis showed it to be insecure at the parameters that our construction requires, even at depth one [2, 38, 26]: the large LTV secret raises the fatigue point to approximately 2^{59}, but a single multiplication with a usable plaintext modulus t forces $q \gtrsim 2^{84}$. We deleted it rather than keeping it as a fast reference, every result transferred bit-exactly, and the homomorphic column of our tables became slower.

A channel of selective failure that passes every functional test. Attaching MACs to the leaves of a bit decomposition by multiplication is unsound. A corrupt party injects an offset δ, the first gate downstream then carries the MAC error δy where y is a secret bit, and consequently the execution aborts when $y = 1$ and completes when $y = 0$. It means that the abort itself carries one bit of the secret.

> **Finding.** Deferring the abort does not close this. Batching hides *when* an abort happens, not *whether* it happens; those are different channels and only the first is closed by batching. We had asserted the opposite before building it.

A crossover withdrawn three times, for three different reasons. The location in Section 9.3 has been wrong three ways: measured against dealt (hence free) preprocessing, so it compared one protocol's online cost with another's total; then measured with Falcon inflated 3.2× by serialisation padding, from which we wrongly inferred that no crossover could exist anywhere; then reported as a

tie because the grid stopped above the boundary. It is at 2–4 $\mathrm{Mbit\,s^{-1}}$. **All three passed our bit-exactness gate and agreed with our cost model**, the first even matching its break-even prediction. A related lesson we now apply: a two-image spot check is not evidence about a wire format—narrowing to 21 bits was exact for the 2^{20} dataset and wrong for *every* image of the 2^{22} one.

Comparison errors of framing. Pricing FHE's *linear* part against MPC's *whole layer* (omitting the ReLU that FHE cannot do at all); computing a crossover against a trusted-dealer baseline (asking whether FHE beats free preprocessing); and the cross-domain triple comparison corrected in Section 6.6.

Honest negatives retained. PIR loses to downloading the database by $\approx 13\times$ with no crossover in N. Hybrid-TFHE does not pay. Faster ring arithmetic does *not* bring FHE close to MPC. Per-layer bit-width tuning was measured at $\approx 10\%$ and not built. A fused garbled max-of-four was measured as a $1.68\times$ *loss* and not built.

12 Threats to validity

In this section we state the threats to the validity of our results.

Quality of the implementation. Our code is research Rust and not tuned C++. Consequently the absolute figures are not competitive, and the ratios measured inside the harness are the defensible output. Where we compare against a published number, as for instance our sign extraction at approximately 0.043 ms against the optimized C++ of Falcon, the difference is one of implementation and not one of protocol, and we say so.

A single machine. All of our figures were obtained on one Apple M1 Pro, and therefore the variance across machines is uncharacterized. The network model is validated on loopback, and consequently the constants for a wide-area network are modelled rather than measured.

Run-to-run variance bounds how sharply a boundary can be read. Re-running an *unchanged* configuration as a separate process moved one cell by 13 %. Repeats *within* one measurement session spread under 2 %, but they share a key-generation session, run back to back and hit warm caches, so they bound jitter rather than reproducibility—our sweep reports the median of three and marks cells whose ranges overlap, and that marking is therefore weaker than it looks. We read every crossover as good to about one grid step, and treat cells within ≈ 13 % of each other as unresolved regardless of which is nominally faster. Latencies carry this variance; message and byte counts are deterministic and reproduce exactly, which is why we prefer them for any claim that has to be sharp.

The simulated network is a model. We validated it to 13.6 % in the worst case, but it remains the expression rounds $\times$ RTT + bytes/bandwidth and not a real wide-area network with congestion, jitter or loss of packets.

The disclosed cryptographic gaps, one of which has since closed. The threshold *key generation* of our homomorphic layer was dealer-based, and we recorded it on the code itself and in the attestation table rather than only in prose. It no longer is, and the reason the dealer was there turns out to be worth more than its removal.

A dealer makes the key components ***uniform*** modulo q while their sum stays ***ternary***, by choosing $s_2 = s - s_0 - s_1$. Both properties at once are what it buys, and no protocol in which each party samples its own component independently can have all three of independent sampling, a small sum, and uniform components. Sampling each s_i uniformly makes s uniform modulo q, and BFV decryption computes $c_0 + c_1 s = \Delta m + (e_1 + e_2 s - eu)$, whose $e_2 s$ term then exceeds $q/2t$: decryption fails outright rather than degrading.

Our key generation takes the other horn. Each party samples s_i ternary and sends it to its predecessor, so the replicated layout (s_i, s_{i+1}) survives — deliberately, because replication is what makes a partial decryption checkable by recompute-and-compare, and a protocol producing only additive shares would remove the dealer at the cost of the malicious-security argument. A single corruption then learns two of the three components, hence $k = s_i + s_{i+1}$, and that $s - k$ is ternary. That is not the weakness it appears to be, because the adversary's residual problem is the one the scheme already rests on:

$$b + ak \;=\; -(a(s-k) + e),$$

an RLWE sample whose secret $s - k$ is ternary. We cannot test hardness, but we can and do test the identity that the reduction rests on — the measured residual has sup-norm exactly 3, the sum of three ternary errors — so a broken reduction would be visibly rather than quietly vacuous. The cost is that $\|s\|_\infty \le 3$ rather than 1, about 1.6 bits of noise budget for a fresh ciphertext and 3.2 for a product, which we report rather than assume because it competes with the smudging budget.

Finding. The expensive part of threshold FHE does not arise here at all. A product decrypts under s^2 because we relinearize nothing, and under replicated sharing each party forms its own share of s^2 *with no communication*: writing $s^2 = \sum_i s_i^2 + 2(s_0 s_1 + s_1 s_2 + s_2 s_0)$, party i holds (s_i, s_{i+1}) and computes $s_i^2 + 2 s_i s_{i+1}$, and the three sum to exactly s^2. It is the same identity replicated multiplication uses, which is no coincidence — it is the same sharing. Distributed generation of a relinearization key, the costly step in threshold FHE, is therefore not deferred but unnecessary, and that is a dividend of the depth-1 design rather than of the key generation.

We measured the cost of using it before proposing to, and we separate two figures that are easy to conflate. Against one local decryption, a threshold decryption is 3.5× in *total work*, which is the throughput cost; the *latency* cost is one party's partial decryption at 1.18×, plus one communication round, because the three partials are computed concurrently on three machines. Quoting the first as a latency figure would repeat the error this paper records elsewhere, so the artifact prints both with that warning and asserts that one party performs between a fifth and a half of the total, which is the premise the parallelism argument needs.

What the input-boundary proof buys, and what it costs. The boundary is the one place where replication cannot help and zero knowledge is unavoidable, and we can now say concretely why. Adding half a Δ to a single coefficient of an input ciphertext — the mildest tamper that does anything at all — turns a computed result of 42 into 43, and two honest co-holders recomputing the evaluation *agree* on the 43, because the evaluation was performed correctly. The fault is in the input, which no amount of recomputation inspects. We draw attention to the off-by-one: a malformed input need not produce a value that a sanity check would reject, and an adversary with more room than half a Δ can choose noise large enough that whether the decryption wraps depends

on the model weights, which turns a correctness failure at the boundary into a channel that leaks what the protocol exists to protect.

The proof itself is implemented: a Schnorr-style Σ-protocol over the ring, made non-interactive by Fiat–Shamir, with Lyubashevsky rejection sampling rather than noise flooding, so that a released response is *perfectly* hiding rather than statistically so. The interesting part is what it costs, and it is not the prover time.

Finding. Soundness slack at the boundary is paid for out of the smudging budget, bit for bit. A Σ-protocol extracts a witness whose norm exceeds the honest one by a slack factor, so a verifier accepting the proof is accepting inputs with noise up to $2^{\text{slack}} \cdot B$; that noise grows through the evaluation and arrives at the threshold decryption, where the flooding that hides the key share must cover it. Our measured 26-bit slack leaves $\lambda = 30$ at the largest modulus this implementation can represent, against the 40 that an honest-input analysis reports. The two analyses were not previously on the same footing, and the gap is entirely this term.

That is why the artifact's attestation for this scheme stays semi-honest: not because the proof is missing, but because it is a tested rather than reviewed implementation, and because $\lambda = 30$ is below what we are willing to call a malicious-security claim. Both are settleable — a tighter proof, or a wider modulus than a `u128` representation admits — and neither is settled here.

What remains open. Two things, neither cryptographic in the sense above. The first is the review and parameter question just described. The second is harness structure: our benchmark evaluates the homomorphic half single-threaded with one decryption key, because simulating one evaluator is cheaper than simulating three, so the reported figures do not yet include the threshold path even though the protocol no longer requires a party that can decrypt.

The Falcon backend's preprocessing used to be the second such gap—dealt from a common seed, so all three parties could reconstruct the mask. It is now generated by bit composition, at a cost of one bit decomposition per element offline. That cost is what the two Falcon rows of Table 16 separate: the cold row pays it on the critical path, the warm row hoists it. An earlier draft of this section said Table 16 measured Falcon's *online* cost while another paragraph said every figure in it was a *total*. Both statements were in the paper at once; the table now has a row for each and says which is which.

A third gap, found while building the pure Falcon route, and closed. Falcon's exact truncation *opens* $c = a + r$. Its stated precondition is that $a + r$ must not wrap, which is a *ceiling* on the mask, and our first Π_{ReLU} satisfied it the cheapest way, with r no wider than the value. That is correct and hides almost nothing: the posterior on a given c collapses to a point at $c = 0$, and for uniform a the leak is $1/(2 \ln 2) \approx 0.72$ bits per truncation, where the batched-bit-decomposition route it replaced opens no function of a secret at all. Making the route "purer" had therefore traded away a security property.

No correctness test could have caught it—leakage is not a wrong answer—and all three tests covering truncation preprocessing happened to pass a mask narrower than the ring, so none ever exercised a wrapping one. The fix is not statistical slack but a *wrap bit*: let r cover the whole ring, making the opened value uniform and *perfectly* independent of a, then correct the resulting overflow with $[c < r]$, which is provably the overflow indicator and is computable from a public c and a secret r for one narrowed `a2b` and one Π_{SS} lift. The alternative—κ bits of statistical slack—needs a ring wider than our narrowed wire encoding permits and measured $1.9\times$ *dearer* in bytes than bit decomposition. The narrow-mask request is now unrepresentable rather than merely documented, and the wrap identity is verified exhaustively over all (a, r) at $\ell = 10$.

A fourth gap, found by attempting the composability proof, and closed. Π_{PC} opens $m \cdot \prod_i c_i$ and reads only whether the result is zero. The c_i carry no randomness of their own — they are determined by the compared bits, the public r and the secret β — so the unit m is the only thing between the transcript and the product, and the comment on our own implementation said exactly that. Our routine for sampling m nevertheless *opened* each candidate in order to reject zeros, and returned the same shares. The justification attached to it read that opening whether a mask is zero leaks nothing, because m is drawn independently of any input and the sampling runs offline. The premise is true and the conclusion does not follow: what the code opened was m itself, and the purpose of m is to blind a *later* opening, so a public m blinds nothing.

The consequence is measurable rather than hypothetical. With m known, $\prod_i c_i = d/m$ is public, and at $\ell = 6$ a single predicate class contains 31 distinct products, so the opening carried $\log_2 31 \approx 5$ bits where the predicate is entitled to one. In Π_{WA} the compared secret is the wrap mask x while $r = a + x$ is public, and therefore those are bits of the activation.

The repair is to test the mask without opening it: draw a second uniform s and open $d = m \cdot s$. Then $d \neq 0$ implies $m \neq 0$, and conditioned on any accepting d the mask is still uniform on $\mathbb{Z}_p^\star$, because each candidate admits exactly one witness. We verify the counting exhaustively over $\mathbb{Z}_p \times \mathbb{Z}_p$ rather than statistically. The cost is one extra random share and one extra batched multiplication per attempt, both offline, and the per-attempt rejection rate rises from $1/p$ to $2/p - 1/p^2$ because either factor may vanish.

We record two things about how this was found. It was not found by testing — every correctness test passed before and after, since leakage is not a wrong answer, and the protocol computed the right comparison throughout. It was found by writing the enumeration that Lemma 3.1 needed, which forced the question "what distribution does the simulator sample this opening from?" at a site where the answer turned out to depend on a secret. Together with the truncation of Section 12 that is two of our nine openings repaired by the same question, which we take as the strongest argument we can offer for stating a composability claim in a form precise enough to be checked.

Breadth of the workload. Our findings are anchored on inference in feed-forward convolutional networks at 28×28. We do not cover transformers, larger images, or workloads other than inference.

This is not an adversarial evaluation. Our tests for the detection of cheating drive real attacks over the transport against the malicious layer, but we remark that this work is not a security audit, and that none of our formal proofs is machine-checked.

13 Open problems

In this section we list the problems that remain open, in the order in which we consider them tractable.

1. **A private convolutional network at the scale of CIFAR** requires SIMD packing in slots, since coefficient packing bounds the size of the image. We remark that the story of composition and security is independent of this choice.

2. **Transformer nonlinearities** (softmax, LayerNorm) need exponential, reciprocal and inverse-square-root subprotocols.

3. **Malicious security at the boundary with homomorphic encryption.** Replication cannot catch a *malformed input ciphertext*, because both honest co-holders compute on it

faithfully and therefore agree. It means that the positioning becomes sharper: a zero-knowledge proof is needed *only at the input boundary and never for the evaluation*, whereas verifiable homomorphic encryption on a single server needs one over the whole computation. We have since implemented the boundary proof, that is, a proof that a ciphertext is a well-formed encryption with bounded noise, and what remains open is a parameter question rather than a construction: the proof's soundness slack is subtracted from the smudging budget bit for bit, so the measured 26-bit slack leaves $\lambda = 30$ at the widest modulus a `u128` representation admits, against the 40 an honest-input analysis reports. Either a tighter proof or a wider modulus recovers it. Composing the proof with the malicious secret-sharing layer, so that a single protocol carries both guarantees, remains open alongside it.

4. **Secret-index array access**, priced by the cost model, before ORAM. CirC [48] implements exactly the two cases either side of that line and does not implement ORAM at all.

5. **Per-instruction protocol assignment**, versus the current per-operation-kind granularity.

6. **Cross-machine and real-WAN validation** of the cost model's constants.

14 Conclusion

Cross-paradigm claims about performance in privacy-preserving computation are not at present falsifiable, because the numbers come from sources that cannot be compared. In this survey we have described a harness in which such claims become falsifiable, and we have reported what changes when the paradigms are re-measured under a uniform accounting.

Three of those changes are worth restating. The crossover between homomorphic encryption and secret sharing belongs to the preprocessing of the baseline rather than to either cryptosystem. Active security costs bytes and not rounds. And the boundary between the comparison protocols lies between 20 and 30 $\mathrm{Mbit\,s^{-1}}$ for a server that pays for its preprocessing on the critical path, and above 100 $\mathrm{Mbit\,s^{-1}}$ for a server that has generated it in advance, which is a difference of a factor of four in bandwidth between two deployments of the same protocol. We mislocated that boundary four times before locating it, and each of the four figures was reproducible and agreed with our own cost model. In Section 9.3 we report that sequence and not only its endpoint, because the failure mode which it illustrates, namely two measurements that are individually sound and mutually incommensurable, is not caught by any check that we had.

We consider that the transferable part of this work is not the artifact but the discipline: identical transports for both legs of any comparison, a cost model checked against a real socket, bit-exactness as a precondition for quoting a cost, and the security level as a machine-checked property of running code rather than as a sentence of prose. Each of these is cheap to adopt, and each of them caught something here that would otherwise have been published.

Reproducibility

Single Rust crate, no GPU. `cargo test` covers correctness; every `bench-*` subcommand emits `-json`. All numbers are produced by these commands; none are hand-entered.

Result	Command
Deep inference, zero bootstraps	`bench-deep-nn -depths 2,4,8,16,32`
Private diagnosis (Table 17)	`bench-medmnist -dataset pneumonia`
Nonlinearity comparison (Table 16, withdrawn)	`bench-medmnist -sweep`
Cost of active security (Table 4)	`bench-malicious -n 32 -bits 32`
Triples: SHE vs OT (Table 7)	`bench-triples`
FHE↔MPC crossover (Table 15)	`bench-layer`
Cost model vs real socket (Section 8.2)	`bench-net`
Proving a quantized neuron (Table 6)	`bench-zkp-layer -widths 1,...,784`
Falcon vs `a2b` vs garbled	`bench-falcon`

References

[1] Martin Albrecht et al. Homomorphic encryption security standard. HomomorphicEncryption.org, Toronto, Canada, 2018.

[2] Martin R. Albrecht, Shi Bai, and Léo Ducas. A subfield lattice attack on overstretched NTRU assumptions. In *CRYPTO*, pages 153–178, 2016.

[3] Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In *EUROCRYPT*, pages 483–501, 2012.

[4] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, pages 420–432, 1991.

[5] Blüml, Huppert, Khayata, Schmidt, and Schneider. SING: Hybrid MPC protocol assignment with graph neural networks. Cryptology ePrint Archive, Paper 2026/1237; to appear, USENIX Security 2026, 2026. `https://eprint.iacr.org/2026/1237`.

[6] Joppe W. Bos, Kristin Lauter, Jake Loftus, and Michael Naehrig. Improved security for a ring-based fully homomorphic encryption scheme. In *IMA International Conference on Cryptography and Coding*, pages 45–64, 2013.

[7] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. CHIMERA: Combining ring-LWE-based fully homomorphic encryption schemes. volume 14, pages 316–338, 2020.

[8] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory*, 6(3), 2014.

[9] Andreas Brüggemann and Thomas Schneider. When trying to catch cheaters breaks the MPC: Breaking and fixing delayed consistency checks in trident, fantastic four, SWIFT, and quad. In *Advances in Cryptology — ASIACRYPT*, 2026. Cryptology ePrint Archive, Paper 2026/234.

[10] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. HyCC: Compilation of hybrid protocols for practical secure computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 847–861, 2018.

[11] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 136–145, 2001.

[12] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. ASTRA: High throughput 3pc over rings with application to secure prediction. In *ACM CCSW*, pages 81–92, 2019.

[13] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. Trident: Efficient 4PC framework for privacy preserving machine learning. In *Network and Distributed System Security Symposium (NDSS)*, 2020.

[14] Edward Chen, Jinhao Zhu, Alex Ozdemir, Riad S. Wahby, Fraser Brown, and Wenting Zheng. Silph: A framework for scalable and accurate generation of hybrid MPC protocols. In *IEEE Symposium on Security and Privacy (S&P)*, pages 848–863, 2023.

[15] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437, 2017.

[16] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In *Advances in Cryptology – CRYPTO 2018*, pages 34–64, 2018.

[17] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[18] Tung Chou and Claudio Orlandi. The simplest protocol for oblivious transfer. In *Progress in Cryptology – LATINCRYPT 2015*, pages 40–58, 2015.

[19] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. SPDZ_{2^k}: Efficient MPC mod 2^k for dishonest majority. In *Advances in Cryptology – CRYPTO 2018*, pages 769–798, 2018.

[20] Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In *Theory of Cryptography (TCC)*, pages 342–362, 2005.

[21] Anders Dalskov, Daniel Escudero, and Marcel Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In *USENIX Security Symposium*, pages 2183–2200, 2021.

[22] Ivan Damgård and Jesper Buus Nielsen. Universally composable efficient multiparty computation from threshold homomorphic encryption. In *CRYPTO*, pages 247–264, 2003.

[23] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, pages 643–662, 2012.

[24] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY – a framework for efficient mixed-protocol secure two-party computation. In *Network and Distributed System Security Symposium (NDSS)*, 2015.

[25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. `https://eprint.iacr.org/2023/1784`.

[26] Léo Ducas and Wessel P. J. van Woerden. NTRU fatigue: How stretched is overstretched? In *ASIACRYPT*, pages 3–32, 2021.

[27] Hendrik Eerikson, Marcel Keller, Claudio Orlandi, Pille Pullonen, Joonas Puura, and Mark Simkin. Use your brain! arithmetic 3PC for any modulus with active security. In *1st Conference on Information-Theoretic Cryptography (ITC)*, pages 5:1–5:24, 2020.

[28] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology — CRYPTO*, pages 823–852, 2020.

[29] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. `https://eprint.iacr.org/2012/144`.

[30] Matthias Felleisen and Daniel P. Friedman. Control operators, the SECD-machine, and the λ-calculus. In *Formal Description of Programming Concepts III*, pages 193–217. North-Holland, 1986.

[31] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *41st ACM Symposium on Theory of Computing (STOC)*, pages 169–178, 2009.

[32] Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium*, pages 809–826, 2022.

[33] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. Extending oblivious transfers efficiently. In *Advances in Cryptology – CRYPTO 2003*, pages 145–161, 2003.

[34] Wen jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *IEEE Symposium on Security and Privacy (S&P)*, pages 1057–1073, 2021.

[35] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium*, pages 1651–1669, 2018.

[36] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1575–1590, 2020.

[37] Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In *Advances in Cryptology – EUROCRYPT 2018*, pages 158–189, 2018.

[38] Paul Kirchner and Pierre-Alain Fouque. Revisiting lattice attacks on overstretched NTRU parameters. In *EUROCRYPT*, pages 3–26, 2017.

[39] Peter M. Kogge and Harold S. Stone. A parallel algorithm for the efficient solution of a general class of recurrence equations. *IEEE Transactions on Computers*, C-22(8):786–793, 1973.

[40] Vladimir Kolesnikov and Thomas Schneider. Improved garbled circuit: Free XOR gates and applications. In *35th International Colloquium on Automata, Languages and Programming (ICALP)*, pages 486–498, 2008.

[41] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. SWIFT: Super-fast and robust privacy-preserving machine learning. In *USENIX Security Symposium*, pages 2651–2668, 2021.

[42] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[43] Jian Liu, Mika Juuti, Yao Lu, and N. Asokan. Oblivious neural network predictions via MiniONN transformations. In *ACM CCS*, pages 619–631, 2017.

[44] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *44th ACM Symposium on Theory of Computing (STOC)*, pages 1219–1234, 2012.

[45] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. DELPHI: A cryptographic inference service for neural networks. In *29th USENIX Security Symposium*, pages 2505–2522, 2020.

[46] Payman Mohassel and Peter Rindal. ABY3: A mixed protocol framework for machine learning. In *ACM CCS*, pages 35–52, 2018.

[47] Eduardo Morais and Ricardo Dahab. Lattice-based cryptography. Technical Report IC-2016-1, Institute of Computing, University of Campinas, April 2016.

[48] Alex Ozdemir, Fraser Brown, and Riad S. Wahby. CirC: Compiler infrastructure for proof systems, software verification, and more. In *IEEE Symposium on Security and Privacy (S&P)*, pages 2248–2266, 2022.

[49] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-party secure inference. In *ACM CCS*, pages 325–342, 2020.

[50] Dragos Rotaru and Tim Wood. MArBled circuits: Mixing arithmetic and boolean circuits with active security. In *Progress in Cryptology — INDOCRYPT*, pages 227–249, 2019.

[51] Microsoft SEAL (release 4.1). `https://github.com/Microsoft/SEAL`, 2023. Microsoft Research, Redmond, WA.

[52] Victor Shoup. NTL: A library for doing number theory. `https://libntl.org/`, 2021.

[53] TFHE-rs: A pure rust implementation of the TFHE scheme. `https://github.com/zama-ai/tfhe-rs`, 2024. Zama.

[54] Sameer Wagh, Divya Gupta, and Nishanth Chandran. SecureNN: 3-party secure computation for neural network training. *Proceedings on Privacy Enhancing Technologies*, 2019(3):26–49, 2019.

[55] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. FALCON: Honest-majority maliciously secure framework for private deep learning. *Proceedings on Privacy Enhancing Technologies*, 2021(1):188–208, 2021.

[56] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[57] Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10(1):41, 2023.

[58] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *27th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 162–167, 1986.

[59] Samee Zahur, Mike Rosulek, and David Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT*, pages 220–250, 2015.